

矿井突水水流漫延模型与逃生方案问题

摘要

矿井水灾是矿山安全开采生产“五大灾害”之首，易造成重大人员伤亡和财产损失。因为矿井水文地质条件复杂，所以矿井水灾事故难以避免。为了降低涉险人员的危险性，减少经济损失，本文针对突出水流漫延过程，以巷道不同漫延方式来建立实时动态模型，研究制定科学救灾方案和最佳逃生路径。

针对问题一，由附件一和附件二，通过 Python 编程分别实现二维平面图和三维立体图，确定出突水点 A1、A2 位置分别在二维平面图和三维立体图的“H0399”巷道。

为研究附件一端点水流首次到达时刻和巷道水流充满时刻，本问建立巷道的流动漫延模型。讨论“铺水”过程中，根据巷道三种突水水流分配方案，分别为没有封闭，没有断流模型；封闭，没有断流模型；没有封闭，断流模型。将三种特殊的模型综合在一起形成平面突水水流漫延模型，通过巷道长度和突水点的突水量求出经过巷道的时间，经过比较求出水流到达端点的首次时间，并将每一个最短时间巷道包含的所有端点看做一个顶点集，直至包含所有端点。巷道突水先后到达 0.1m 后，再同时逐步上升到 0.3m，直至上升到 3m 灌满整个巷道。通过上述模型，得到以下结论：（1）铺满巷道时间为 1672.06min；（2）所有巷道水位高度到达 0.3m 的时间为 5016.18min；（3）灌满所有巷道的时间为 5016.80min。

为研究附件二端点水流首次到达时刻和巷道水流充满时刻，首先筛选出等于和低于突水位置的端点，并利用上述模型，求出形成平面集合包含的所有端点，集合中的端点已铺满 0.1m，以此找出筛选出但未在集合中的端点，采用 BFS 广度优先模型搜索这些端点对应巷道的另一个端点。接着将模型简化为以形成的平面为基础，从低到高，突水以平行于平面方向向上漫延，直至到达 3m，灌满整个巷道。在整个过程中，铺满和灌满同时精选，且灌满整个巷道的的时间就是到达最后一个端点的时间，时间均为 50249.91min。

针对问题二，由条件可知，在突水 1 分钟后所有矿工都在无突水水流巷道。经分析可知当巷道内水面高度超过 0.3m 时，矿工均逃生成功。根据问题一得到所有端点随时间水位变化情况，根据端点水流到达时刻的先后顺序确定顺水和逆水方向。以到达邻点所需要的时间为权重，采用实时动态网络的 Dijkstra 算法，设计三位矿工最佳逃生路径，具体逃生路径和时刻表分别保存到附件 2、result2-1.xlsx 和 result2-2.xlsx 中。

针对问题三，为研究矿井有两个突水点发生突水时，各端点水流首次到达时刻和巷道水流充满时刻，本问采用实时动态的流动漫延模型，因为当来自不同突水点的两个漫延路径相互碰撞时，就会形成封闭，此时按照模型会将分给该巷道的流量回到上一端点重新分配。具体结果分别保存到文件 result3-1.xlsx 和 result3-2.xlsx 中。

针对问题四，经分析得知前 3 分钟矿工逃生路径与附件 2 相同，5 分钟后调整有两个突水口时的最佳逃生路径，具体结果分别保存到文件 result4-1.xlsx 和 result4-2.xlsx 中。

关键词：流动漫延模型 BFS 广度优先模型 最佳逃生路径 实时动态网络的 Dijkstra 算法

一、问题重述

1.1 问题背景

矿井水灾是矿山安全开采生产“五大灾害”之首，易造成重大人员伤亡和财产损失。因为矿井水文地质条件复杂，所以矿井水灾事故难以避免。一旦发生突水，极易增加涉险人员危险性和造成经济损失。

矿井巷道系统根据矿藏分布和矿脉走向布局，形成复杂。

当水灾发生时，若能快速推演出突水水流蔓延过程制定科学的逃生方案，就能最大程度地减少损失。

矿井巷道系统特征和条件：

- 1、矿井巷道系统根据矿藏分布和矿脉走向布局，形成复杂的交叉三维网络结构。
- 2、巷道断面多种多样，本文简化为统一的矩形断面（宽 4m，高 3m），断面底边与水面平行且两端断面底边中点的连线表示一段巷道。
- 3、突水水流以 0.1 的初始水位向前蔓延，初水水位为水流首次到达后能够保持的高度。
- 4、当水流蔓延到巷道的分叉节点处时，水流向水平巷道和下行巷道平均分流，且初始水位不变
- 5、以开始突水时间为零时刻，各突水点的突水量均为 $30\text{ m}^3/\text{min}$ 。

1.2 问题提出

问题一：已知端点水流到达时刻是指突水水流首次流经该点的时刻，巷道充满水时刻是指巷道中水流的水平面达到巷道最高点的时刻。文中给出了附件一的突水点位置 A1 (5349.03,4931.90,10.00)，附件二的突水点位置 A2 (4143.12,4376.28,6.33) 等条件，要求分析巷道的某一点发生突水的水流过程，建立突水水流在巷道的流动蔓延模型，并分别给出网络中各巷道水流的变化情况，结果均保留两位小数。

问题二：假设当矿井发生突水后，安全生产部门即刻监控到突水情况且当在突水 1 分钟时发布逃生通知。已知工人在无突水水流巷道时，行进速度为 4m/s；巷道内水面高度为 0.3m 时，工人逆水行进速度为 1m/s，顺水行进速度为 2m/s；巷道内水面高度超过 0.3m 时，不建议涉水通行。文中给出了附件 1 中的出入口位置分别为 (3252.16,3326.63,10.00)，(3173.10,2819.97,10.00)，矿工的位置分别为 (5808.18,5367.75,10.00)，(5194.00,4785.31,10.00)，(6190.81,3434.29,10.00)，附件 2 中的出入口位置分别为 (6336.99,6073.22,36.15)，(6416.05,6579.88,8.69)，矿工的位置分别为 (4395.15,4614.53,6.59)，(3398.34,5965.56,1.31)，(3879.44,4125.47,6.22) 等条件。要求根据问题一中所建流水蔓延模型，协助安全生产部门为各矿工设计最佳逃生路径。

问题三：文中给出了附件一中第二个突水点位置 B1 (3760.40,3808.33,10.00)，附件二的突水点位置 B2 (5883.14,5643.35,40.37) 等条件。已知 B1 在 A1 点突水 4 分钟后开始突水，B2 在 A2 点突水 5 分钟后开始突水。要求分析矿井有两个突水点发生突水的水流过程，建立突水水流在巷道的流动蔓延模型并给出网络中各巷道水流变化情况。

问题四：已知当矿井出现第二个突水点后，安全生产部门即刻监控到突水情况。假设在第二突水点突水 1 分钟后，安全生产部门发布调整后的逃生方案。要求分析各矿工调整后的最佳逃生路径。

二、问题分析

问题一：根据条件可知，附件一形成二维平面图，附件二形成三维立体图，突水点 A1、A2 位置分别在二维平面图和三维立体图的“H0399”巷道。为研究端点水流到达时刻和巷道水流充满时刻，针对附件一，本问建立三种特殊突水水流在巷道的流动漫延模型。分别为没有封闭，没有断流模型；封闭，没有断流模型；没有封闭，断流模型。将三种特殊的模型综合在一起形成平面突水水流漫延模型，在模型基础上求出水流到达端点的最短时间，并将每一个最短时间巷道包含的所有端点看做一个整体，形成集合，直至包含所有端点。巷道突水先到达 0.1m，再同时逐步上升到 0.3m，直至到 3m 灌满整个巷道。针对附件二，首先筛选出等于和低于突水位置的端点，并利用上述模型，求出形成平面集合包含的所有端点，以此找出筛选出但未在集合中的端点，采用 BFS 方式广度搜索这些端点对应巷道的另一个端点。接着将模型简化为以形成的平面为基础，从低到高，突水以平行于平面方向向上漫延，直至到达 3m，灌满整个巷道。

问题二：为制定出每个矿工的最佳逃生路径，本问采用实时动态网络的 Dijkstra 算法为矿工设计从矿工的初始位置到安全出入口的最短路径，其中以到达端点所需要最小时间为权重。根据问题一可知突水水流到达每个端点的最小时间，每个巷道突水水流方向以及巷道之间漫延方向。由此可判断矿工在不同巷道的逃生前进速度，求出矿工穿过不同巷道对应的时间，以最小时间作为权重，最后采用实时动态网络的 Dijkstra 算法求出每个矿工的最佳逃生路径。

问题三：由条件可知，突水点 A1 突水 4 分钟后突水点 B1 开始突水，突水点 A2 突水 5 分钟后突水点 B2 开始突水。为研究矿井有两个突水点发生突水时，各端点水流到达最短时间和巷道水流充满时间，本问采用与问题一相同的模型，因为当来自不同突水点的两个漫延路径相互碰撞时，就会形成封闭，此时按照模型会将分给该巷道的流量按比例分给其他巷道，再次回到各自轨道上按模型漫延。因此得到结论为：突水到达端点的最短时间与突水点单独突水最短时间相同，互不影响。但是，来自两个突水点的不同路径的突水在同一巷道相互作用时，会增加经过该巷道的速度，所以将所有巷道铺满至 0.1m 的总时间和灌满整个巷道的总时间会比一个突水点时总时间的一半还短。

问题四：经分析得知前 4 分钟矿工逃生路径与问题二相同，4 分钟后第二突水点开始突水，但此时工人并不知道此时有两个突水点开始突水，直至 5 分钟后调整有两个突水口时的最佳逃生路径，并通过实时动态网络的 Dijkstra 算法求出每个矿工在两个突水点开始突水时的最佳逃生路径。

三、模型假设

- 1、假设巷道断面为矩形断面，断面底边与水平面平行。
- 2、假设当水流漫延到巷道的分叉节点处，水流向水平巷道和下行巷道平均分流。
- 3、假设以开始突水时间为零时刻，各突水点的流量均为 $30\text{m}^3/\text{min}$ 。
- 4、假设端点水流到达时刻是指突水水流首次流经该点时刻，巷道充满水时刻是指巷道中水流的水平面达到巷道最高点的时刻。
- 5、假设工人反应时间为零。
- 6、假设巷道内水面高度超过 0.3m 时，逆流方向路径中断。

四、符号说明

符号	含义	单位
Q	突水点突水量	m^3 / min
m_i	第 i 个端点名称	
m_j	第 j 个端点名称	
$V_{m_i m_j}$	巷道在高 0.1m 时对应体积	m^3
$m_i m_j$	端点 m_i 和 m_j 组成的巷道名称	
t_0	表示所有端点铺满时刻	min

五、模型建立与求解

本文矿井突水漫延模型整体思维导图如下：



图 1 矿井漫延思维导图



图 2 模型思维导图

5.1 问题一模型建立与求解

5.1.1 问题一模型建立

本问运用 Python，画出附件一中每个端点、各个端点组成的巷道网络图以及用五角星标出突出点 A1 位置，如下图：

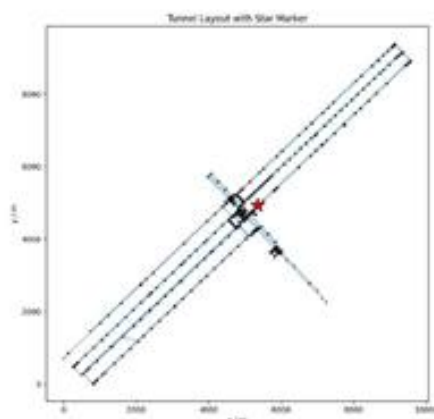


图 3 巷道平面示意图

如图所示：附件一所形成的巷道为二维平面图。由此可知，突水水流先以 0.1m 初始水位向前漫延，直到铺满全部巷道，全部巷道一起再从 0.1m 逐渐上升至 3m，灌满整个赛道。^[1-5]

本问运用 Python，画出附件二中每个端点、各个端点组成的巷道网络图以及用五角星标出突出点 A2 位置，如下图：

Mine network with exits/miners/source (scaled down)

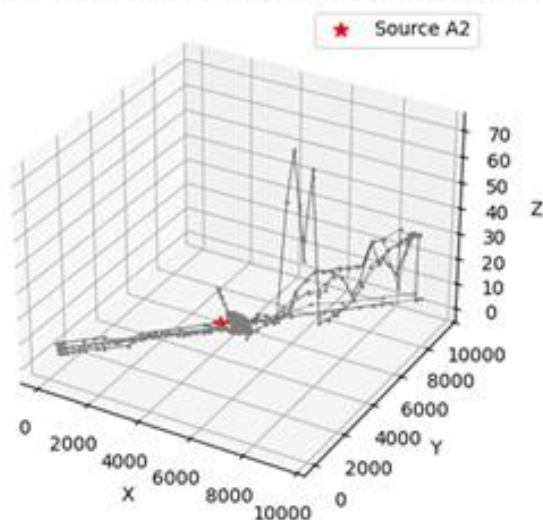


图 4 三维巷道示意图

如图所示：附件二所形成的巷道为三维立体图。由此可知，突水水流从突水点以 0.1m 水位先向同一高度和比突水点高度低的位置漫延，待达到铺满 0.1m 最大面积时，将所有端点形成一个平面，以这个平面为底且以平面方向平行向上漫延，直至巷道突水水流达到 0.3m，3m。由于三维立体图有落差，所以铺和盖的是同时进行的。^[6-10]

5.1.1.1 寻找距离突水点最近的巷道

对于附件一平面图形：

运用 Python，通过向量投影法计算突水点到巷道的最短距离，结果为突水点 A1 刚好在 H0399 巷道上，并且通过距离公式 $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$ 得出突水点 A1 距离巷道两端点 P0351，P0358 距离相同。

为了确保位置准确性，本问采用数学表达式进行验证。

给定两点 $P0351(x_1, y_1) = (5205.06, 4795.87)$ ， $P0358(x_2, y_2) = (5492.97, 5067.92)$

斜率为：

$$k = \frac{y_2 - y_1}{x_2 - x_1} = \frac{5067.92 - 4795.87}{5492.97 - 5205.06} = \frac{272.05}{287.91} \approx 0.9449 \quad (1)$$

直线方程为：

$$y = 0.9449x - 122.09 \quad (2)$$

将 A1 (5349.03, 4931.90) 带入直线方程得 $y \approx 4932.83$ 。

故误差为 $4932.83 - 4931.90 = 0.93$ 。虽 A1 不在巷道所在直线上，但误差很小，所以可以看做 A1 就在巷道所形成的直线上。

综上所述：突水点 A1 在巷道 H0399 终点位置，开始从巷道中点以 $30\text{m}^3/\text{min}$ 突水量同时向两端点 P0351，P0358 漫延，并经过 5.28min 同时到达两端点。

对于附件二三维立体图形：

运用 Python，通过向量投影法计算突水点到巷道的最短距离，结果为突水点 A2 刚好也在 H0399 巷道上，此时 A2 距离巷道两端点 P0351，P0358 距离也相同。

为了确保位置准确性，采用和上述相同的方式进行验证。

设直线参数方程为

$$\begin{cases} x = 4384.08 - 287.90t \\ y = 4603.97 - 272.04t \\ z = 6.59 - 0.31t \end{cases} \quad (3)$$

其中 $t \in \mathbb{R}$ ，将突水点 A2 (4143.12, 4376.28, 6.33) 带入参数方程得

$$\begin{cases} t_1 = 0.8368 \\ t_2 = 0.8369 \\ t_3 = 0.8387 \end{cases} \quad (4)$$

由于三个 t 值非常接近，可认为在误差范围内一致，可看做 A2 在巷道所形成的直线上。

5.1.1.2 附件一平面模型的分类讨论

$G(V, E)$ ：巷道网络图， V 为节点集 ($|V| = n$)， E 为边集。

$U(m_j)$ 表示邻点集

$$e_{m_i, m_j} \in E$$

$$m_i \in U(m_j)$$

$$\min U(m_j) = \{m_i, m_k\}, k \neq i, k \neq j, k \in (1, 645) \quad (5)$$

模型一（没有封闭，没有断流）：

$$Q_{m_i, m_j} = \frac{Q_{m_i, m_j}}{n-1} \quad (6)$$

$$t_j = \frac{V_{m_i m_j}}{Q_{m_i m_j}} = \frac{SL_{m_i m_j}}{Q_{m_i m_j}} \quad (7)$$

$$t_{\min} = \min(t_j) \quad (8)$$

其中 S 为巷道的截面，巷道突水未铺满前 $S = 0.4m^2$ 。

找出最小时间的巷道包含的所有端点，所有端点形成一个集合，做为一个整体，以此类推。

以下图为例：

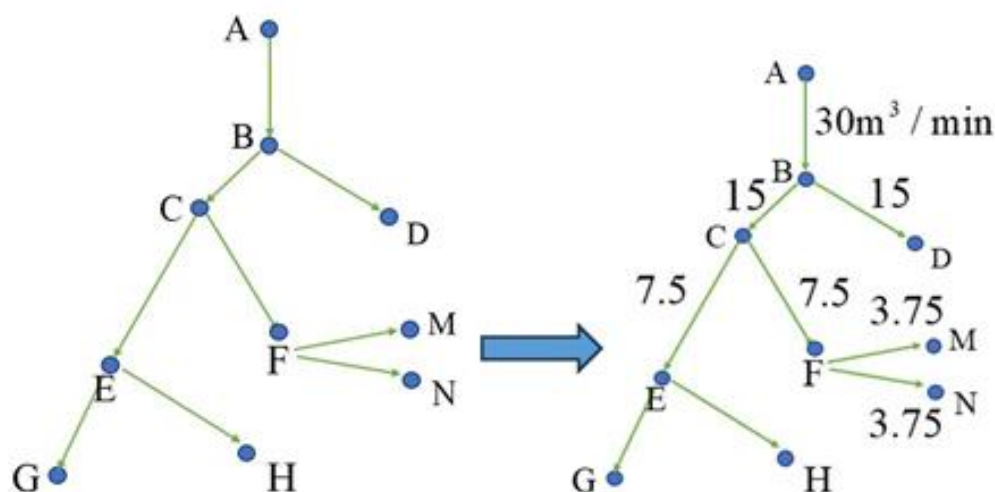


图 5 示意图

第一轮：假设 $Q_{AB} = 30m^3 / min$ ，则 $Q_{BC} = Q_{BD} = \frac{Q_{AB}}{2} = 15m^3 / min$ 。若 $\frac{SL_{BC}}{Q_{BC}} < \frac{SL_{BD}}{Q_{BD}}$ ，此时 BC 巷道优先铺满，故可将 A、B、C 看做一个集合且 BC 巷道向外漫延。

第二轮： $Q_{CE} = Q_{CF} = \frac{1}{2}Q_{BC} = 7.5m^3 / min$ 。设 CE 巷道铺满 0.1m 时间最短，则

$\frac{SL_{CE}}{Q_{CE}} < \frac{SL_{CF}}{Q_{CF}}, \frac{SL_{CE}}{Q_{CE}} < \frac{SL_{BD}}{Q_{BD}}$ ，故可将 A、B、C、E 一个集合且 BC 巷道向外漫延。

第三轮： $Q_{EG} = Q_{EH} = \frac{Q_{CE}}{2} = 3.75m^3 / min$ ， $Q_{FM} = Q_{FN} = \frac{Q_{CF}}{2} = 3.75m^3 / min$ 。设 FM 巷道铺满 0.1m 时间最短，故可将 A、B、C、E、F 集合且 BC 巷道向外漫延。以此类推，向后漫延，直到不能漫延。

模型二（没有封闭，断流）： $t_{\min}$ 时对应的巷道 $m_i m_j$ 后无巷道，则形成断流。

当 $n-2 > 0$ 时：除 $t_{\min}$ 时对应的巷道 $m_i m_j$ 外其他从 m_i 端点流出的巷道对应的突出流量为

$$Q = \frac{Q_{m_i m_j}}{n-1} * \frac{n-1}{n-2} = \frac{Q_{m_i m_j}}{n-2} \quad (9)$$

当 $n-2=0$ 时：考虑 m_k 是否有除 $m_k m_l$ 巷道外从 m_k 流出的巷道，若有，就以该方法按比例分流；若没有，则继续前推比 m_l 上一个加入集合的端点，若一直没有，则结束循环。

下图为例：

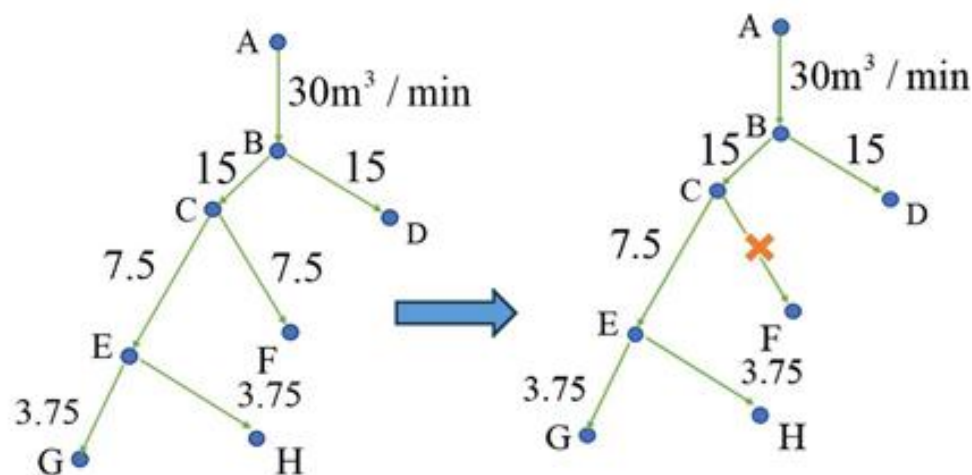


图 6 示意图

假设突水蔓延情况与模型一相同，则第一轮，第二轮相同，第三轮时巷道 CF 断流，可将其巷道看做消失，此时 $Q_{EG} = Q_{EH} = \frac{Q_{CE}}{2} + \frac{Q_{CF}}{2} = 7.5 \text{ m}^3 / \text{min}$ 。

模型三（封闭，没有断流）：

假设 $Q_{AB} = 10 \text{ m}^3 / \text{min}$

(1) 假设突水从端点 C 流出的巷道只有 CD，从端点 D 出的巷道只有 CD，以下面图形为例：

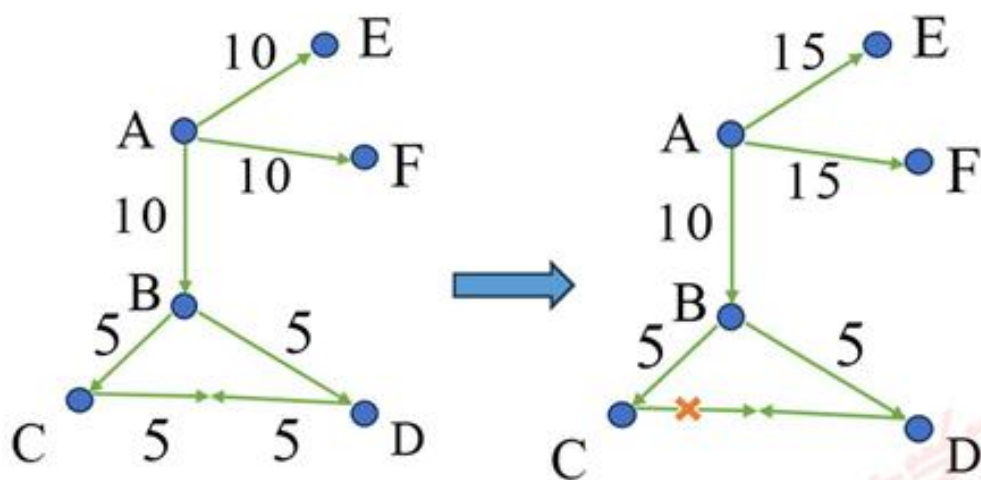


图 7 示意图

此时 $Q_{BC} = Q_{BD} = 5\text{m}^3/\text{min}$ ，如果 $t_{BC} < t_{BD}$ ，则 BC 巷道先铺满 0.1m，突水先从 BC 巷道漫延到 CD 巷道，待 BD 巷道铺满 0.1m 时，两个方向突水共同作用，直到 CD 巷道铺满 0.1m。因为突水从端点 C 流出的巷道只有 CD，所以返回上一端点 B，而 B 端点只和 BC、BD 两巷道连接，故继续返回上一端点 A，存在除 BC 巷道外从 A 点流出的巷道 AE、AF，此时对其平均分流， $Q = Q_{原} + Q_{分} = 10 + 5 = 15\text{m}^3/\text{min}$ 。

(2) 假设突水从点 C 流出的巷道为 CD、CG，从端点 D 出的巷道只有 CD，以下面图形为例：

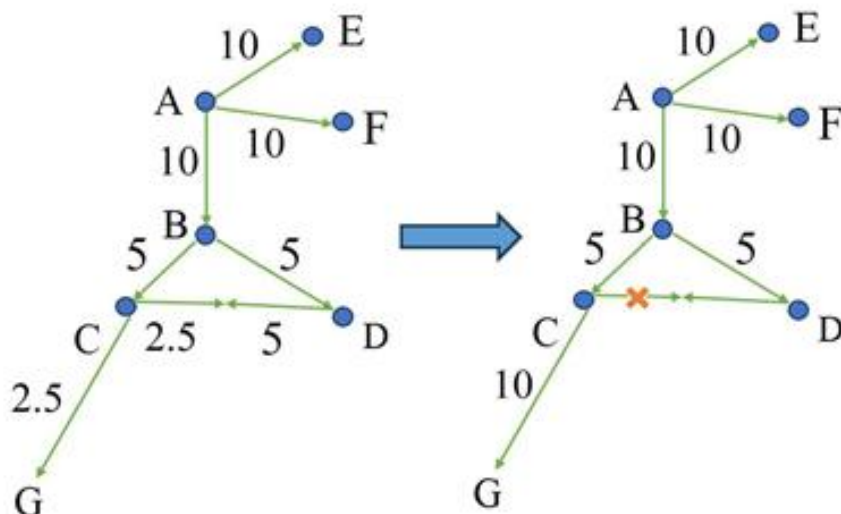


图 8 示意图

CD 巷道突水铺满 0.1m 方式和上述相同，若 CD 巷道比 CG 巷道先铺满 0.1m，此时 $Q_{CG} = Q_{CD} + Q_{CG原} = 7.5 + 2.5 = 10\text{m}^3/\text{min}$ ，CD 突水分流到 CG 巷道。若 CD 巷道比 CG 巷道后满 0.1m，将 CG 巷道突水分流到 CD 巷道，再按 (1) 模型推进。

将三种模型情况综合考虑，先遇到模型中的哪种情况，就先考虑哪种模型，解决到达端点的最短时间。

5.1.1.3 附件二立体模型的分类讨论

首先筛选出附件二 Excel 表格中和突水点 A2 位置相同和比突水点 A2 位置低的数据。以筛选出来的端点为基，采用与平面一相同的模型和方法，将可以形成集合的端点看做一个平面。若筛选出来的端点未在集合内，则通过 BFS 路径搜索算法寻找与这些端点连接成巷道的另一个端点。接着将模型简化为以形成的平面为基础，从低到高，突水以平行于平面方向向上漫延，直至到达 3m，灌满整个巷道。

5.1.2 问题一模型求解

附件一：所有端点完全铺满（高度达到 0.1m）时间为 1672.06min，所有巷道高度达到 0.3m 时时间为 5016.18min，所有巷道完全灌满（高度达到 3m）时间为 50161.80min。

附件二：得出的结论为灌满整个巷道的的时间就是到达最后一个端点的时间，所有巷道完全铺满时间为 50249.91min。

针对所有端点完全铺满时间为 1672.06min，而不是 1674.09min 解析：

铺满所有巷道时间如下：

$$T = \frac{V}{Q} = \frac{0.1 \times 4 \times L}{30} = \frac{0.1 \times 4 \times 125557.1}{30} = 1674.09 \text{ min}$$

铺满所有端点的时间比铺满所有巷道的时间短原因是当所有端点都遇到水时，存在一些巷道还没有水。图形如下：

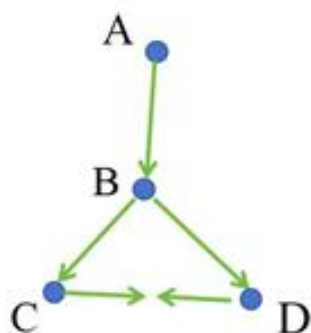


图 9 示意图

5.2 问题二模型建立与求解

5.2.1 问题二模型建立

为了给每个矿工制定最佳逃生路径，本问运用 Python，画出附件一中每个端点、各个端点组成的巷道网络图、突出点 A1 以及两个出入口位置 $P_1(3252.16, 3326.63, 10.00)$ ， $P_2(3173.10, 2819.97, 10.00)$ 和三个矿工位置 $P_3(5808.18, 5367.75, 10.00)$ ， $P_4(5194.00, 4785.31, 10.00)$ ， $P_5(6190.81, 3434.29, 10.00)$ ，图形如下：

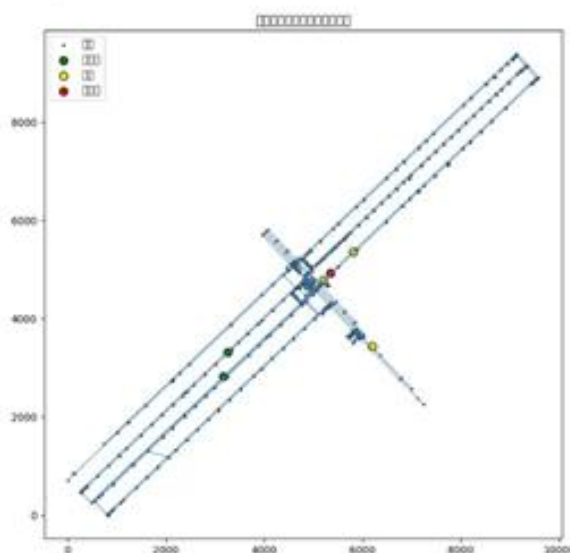


图 10 最佳逃生路径

运用 Python，画出附件二中每个端点、各个端点组成的巷道网络图、突出点 A2 以及两个出入口位置 $P_6(6336.99, 6073.22, 36.15)$ ， $P_7(6416.05, 6579.88, 8.69)$ 和三个矿工位

置 $P_8(4395.15, 4614.53, 6.59)$, $P_9(3398.34, 5965.56, 1.31)$, $P_{10}(3879.44, 4125.47, 6.22)$, 图形如下:

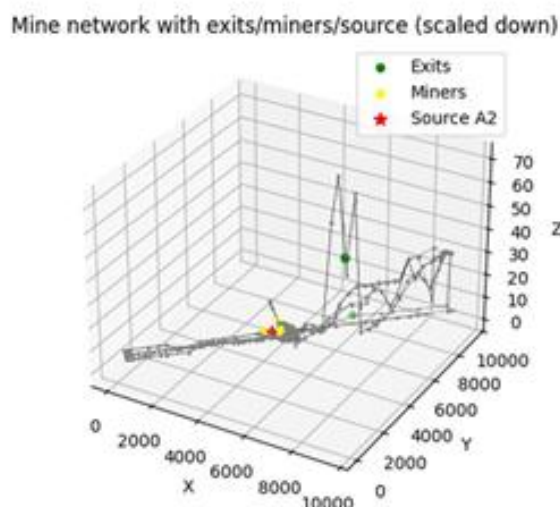


图 11 巷道网络图

在矿井发生突水灾害后, 为各矿工设计最佳逃生路径是一个动态网路最短路问题, 本问采用 Dijkstra 算法为矿工设计从矿工的初始位置到安全出入口的最短路径。其中以到达端点所需要的最小时间为权重, 但考虑到边权是时刻变化的, 且该值取决于水流到达该巷道的时间和水深。本文将逃生时间线离散为多个时间步, 并把每个时间步看成一个集合, 在集合内网络状态不变, 便将动态网络问题转化为静态问题, 从而达到 Dijkstra 算法实现。

对于附件一, 运用 MATLAB, 可分析得出入口 P_1 在“P0393”端点, 出入口 P_2 在“P0526”端点。矿工 P_3 在距离“P0661”端点 35.1m, 矿工 P_4 在距离“P0351”端点和“P0349”端点 15.3m, 矿工 P_5 在距离“P0016”端点 245.73m。

对于附件二, 运用 MATLAB, 可分析得出入口 P_6 在“P0393”端点, 出入口 P_7 在“P0526”端点。矿工 P_8 在“H0389”巷道, 距离“P0351”端点和“P0349”端点 15.3m; 矿工 P_9 在“H0012”巷道, 距离“P0016”端点和“P0017”端点 245.7m; 矿工 P_{10} 在“H0968”巷道, 距离“P0660”端点 99.76m。

首先运用传统的 Dijkstra 算法, 以距离为权重, 算出每个矿工到出入点的最短距离。将最短距离和最小速度乘积与突水水流高度到达 0.3m 时的时间进行比较。经计算可得, 在突水水流高度达到 0.3m 时, 矿工不可能还在巷道内。

6.2.1.1 时间权重的计算

第一轮: 因为突水 1 分钟时发布逃生通知, 所以要先找出 1 分钟突水能够到达的端点。本问采用 BFS 广度优先模型, 找出与矿工位置形成巷道的另一个端点, 并求出矿工以 4m/s 前进速度时到达此端点时间 t

$$t = \frac{L_m}{4} \quad (10)$$

其中 L_m 表示巷道的长度, 根据 result-1.xlsx 可知当 $T = \frac{L}{4} + 1$ 时有突水水流的端点, 由此判断 BFS 搜索的端点是否有突水水流。经检验可知, 与矿工位置形成巷道的另一个

端点都没有突水。所以只需找到最短巷道，此巷道就是该矿工逃生的第一段路径并将时间作为该路径的权重。将组成巷道的两个端点看作整体，形成一个集合 A。

第二轮：BFS 广度优先模型，找出与集合 A 中所有端点形成巷道的另一个端点，并求出矿工以 4m/s 前进速度时到达此端点时间 $t_1 = \frac{L_m}{4}$ 根据 result-1xlsx 可知当 $T_1 = T + t_1$ 时有突水水流的端点。

由问题一结果可知突水水流到达每个端点的的最小时间，每个巷道突水水流方向以及巷道之间蔓延方向。

若有突水水流的端点在 BFS 搜索的端点：

1、当矿工行进方向与巷道突水水流方向相同时：矿工为顺水行进，矿工经过巷道时间为 $\frac{L_m}{2}$ 。

2、当矿工行进方向与巷道突水水流方向相反时：矿工为逆水行进，矿工经过巷道时间为 L_m 。

若有突水水流的端点不在 BFS 搜索的端点：矿工经过巷道时间为 $\frac{L_m}{4}$ 。

通过比较求出最短时间，将最短时间的巷道作为逃生路径并将此时时间看作该路径的权重。将组成巷道的所有端点看作整体，形成一个集合 B。

接着与上述模型相同，以此类推，直到集合中包含出入点结束。加入集合的先后顺序就是矿工的最佳逃生路径。

6.2.1.2 基于时间动态网络 Dijkstra 算法步骤

$$\begin{aligned}
 \min \quad & T_{jk}(t) = \frac{L_{jk}}{v_{jk}(h(e_{jk}, t))} \\
 \text{s.t.} \quad & \sum_{k: e_{km} \in E} \sum_{t \in T} x'_{mk} = 1 \\
 & \sum_{j: e_{jn} \in E} \sum_{t \in T} x'_{jn} = 1 \\
 & \sum_{j: e_{ju} \in E} \sum_{t \in T} x'_{ju} = \sum_{k: e_{uk} \in E} \sum_{t \in T} x'_{uk}, \quad \forall u \in V \setminus \{s, d\} \\
 & \sum_1^T Q = 30m^3 / \min \\
 & T_{\text{总}} \leq 50222 \\
 & t_0 \leq 1674.06
 \end{aligned} \tag{11}$$

t_0 表示所有端点铺满时刻

其中 L_{jk} ：巷道 e_{jk} 的长度 (m)

$\tau_{jk}(t)$ ：表示在 t 时刻开始通过巷道 e_{jk} 所需的时间 (s)

T ：连续决策变量，表示工人到达目标出口 d 的总时间 (s)。

左边 $\sum_{j \in \mathcal{N}_u} \sum_{t \in \mathcal{T}} x_{ju}^t$: 计算所有流入中间节点 u 的“流”的总和。

右边 $\sum_{k \in \mathcal{N}_u} \sum_{t \in \mathcal{T}} x_{uk}^t$: 计算所有流出中间节点 u 的“流”的总和。

$\forall u \in V \setminus \{s, d\}$: 该约束对网络中除起点和终点之外的所有节点 u 都必须成立。

5.2.2 问题二模型求解

基于上述算法,进一步运用 Dijkstra 算法,将矿工位置 P_3 近似为在“P0661”端点,将矿工位置 P_4 近似为在“P0351”端点,将矿工 P_6 位置近似为在“P0016”端点。

最后通过算法得出矿工:

从位置 P_3 到达最近出入口的最短路径为: 662 427 425 661 659 357 352 350 347 345 658 653 651 652 53 338 641 625 317 316 430 313 315 314 637 635 193 195 645 644 419 420 170 169 649 160 151 147 150 153 520 518 395 392 393, 最短路径总长度: 3718.009m

从位置 P_4 到达最近出入口的最短路径为: 349, 348, 332, 330, 331, 52, 327, 627, 319, 321, 416, 648, 185, 181, 182, 631, 170, 169, 649, 160, 151, 147, 150, 153, 520, 518, 395, 392, 393, 最短路径总长度: 2665.6m

从位置 P_5 到达最近出入口的最短路径为: 17, 24, 22, 23, 121, 126, 369, 606, 607, 611, 612, 370, 139, 137, 134, 133, 136, 201, 561, 205, 365, 367, 342, 566, 565, 161, 155, 163, 164, 158, 149, 146, 148, 150, 153, 520, 518, 395, 392, 393, 最短路径总长度: 3859.1m

从位置 P_8 到达最近出入口的最短路径为: 349, 348, 332, 330, 331, 52, 327, 627, 319, 321, 416, 648, 185, 181, 182, 631, 170, 169, 649, 160, 151, 147, 152, 153, 520, 518, 395, 392, 393, 最短路径总长度: 2672.2m

从位置 P_9 到达最近出入口的最短路径为: 从位置 P_6 到达最近出入口的最短路径为: 17, 24, 22, 23, 123, 126, 369, 606, 607, 611, 612, 372, 139, 137, 134, 133, 136, 201, 561, 205, 365, 367, 342, 566, 565, 161, 155, 163, 164, 158, 149, 146, 148, 150, 153, 520, 518, 395, 392, 393, 最短路径总长度: 3865.9m

从位置 P_{10} 到达最近出入口的最短路径为: 660, 662, 661, 659, 357, 352, 350, 347, 345, 658, 653, 651, 652, 53, 338, 641, 625, 317, 316, 430, 313, 315, 314, 637, 635, 193, 195, 645, 644, 419, 420, 170, 169, 649, 160, 151, 147, 150, 153, 520, 518, 395, 392, 393, 最短路径总长度: 3675.629m

5.3 问题三模型建立与求解

5.3.1 问题三模型建立

运用 Python, 针对附件一, 画出两个出入口, 三个矿工位置以及两个突水点位置, 如下图:

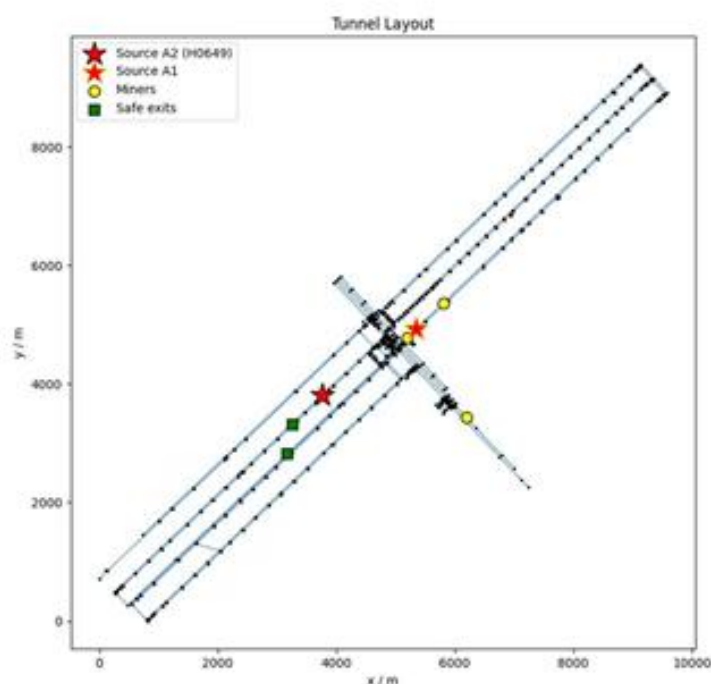


图 12 巷道网络图

运用 Python，针对附件二，画出两个出入口，三个矿工位置以及两个突水点位置，如下图：

Mine network with exits/miners/sources A1&A2

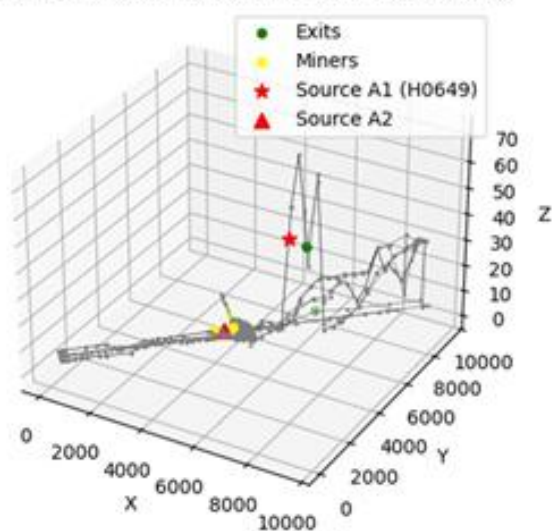


图 13 巷道网络图

本问采用与问题一相同的模型，在突水点 A1 突水 4 分钟后突水点 B1 开始突水，在突水点 A2 突水 5 分钟后突水点 B2 开始突水。两者分别以问题一的模型漫延，到达端点的最短时间与突水点单独突水最短时间相同，互不影响。因为当来自不同突水点的两个漫延路径相互碰撞时，就会形成封闭，此时按照模型会将分给该巷道的流量按比例分给其他巷道，再次回到各自轨道上按模型漫延。但是，来自两个突水点的不同路径的

突水在同一巷道相互作用会增加经过该巷道的时间，所以将所有巷道铺满至 0.1m 的总时间和灌满整个巷道的总时间会比一个突水点时总时间的一半还短。

例如：

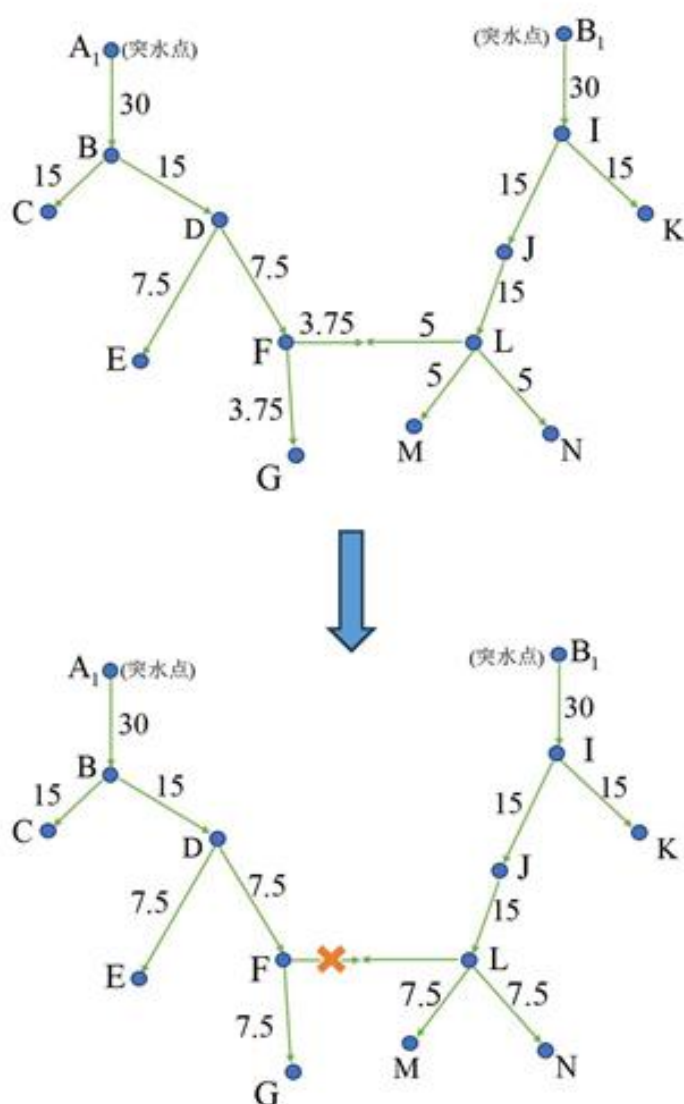


图 14 示意图

假设经过三轮漫延后，集合 $X = \{A_1, B, D, F\}$, $Y = \{B_1, I, J, K, L\}$ ，前三轮两个突水点漫延路径互不影响，到第四轮时，巷道 DF 和巷道 JL 漫延的突水会在巷道 FL 相互叠加，但这属于封闭模型，根据模型漫延方法，巷道 FL 铺满 0.1m 后可看做不存在，将突水量按各自比例平均分配。所以巷道 FG 突水量从 $3.75\text{m}^3/\text{min}$ 重新变为 $7.5\text{m}^3/\text{min}$ ，巷道 LM, LN 突水量从 $5\text{m}^3/\text{min}$ 重新变为 $7.5\text{m}^3/\text{min}$ 。与单独一个突水点突水流量相同，故又在各自轨道上漫延，所以可以将本文看做两个突水点各自在各自轨道上漫延，不影响到达各端点的最小时间和铺满整个巷道的的时间。

5.4 问题四模型建立与求解

针对附件一：开始时突水点 A1 开始突水，但 1 分钟后矿工才接受到逃生通知，1 分钟到第 4 分钟，矿工开始矿工逃生，路径与问题二相同。4 分钟后第二突水点开始突水，但此时工人并不知道此时有两个突水点开始突水，直至 5 分钟后调整有两个突水口时的最佳逃生路径，并通过时动态网络的 Dijkstra 算法求出每个矿工在两个突水点开始突水时的最佳逃生路径。图形如下：

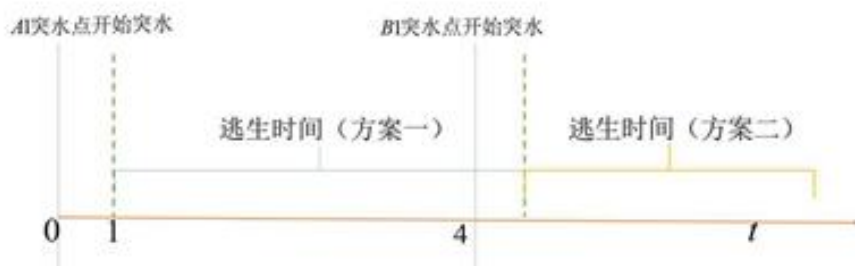


图 15 示意图

结果显示：在附件一的二位平面图中，本问三个矿工逃生路径与问题二最短时间基本相同。在附件二的空间立体图中，本问两个矿工逃生路径与问题二基本相同，只有一个特殊矿工受到第二突水点的影响较大，通过时动态网络的 Dijkstra 算法求出该矿工在两个突水点开始突水时的最佳逃生路径。

六、模型的优缺点

- (1) 在传统的 Dijkstra 算法下，以动态用时对边赋权，建立了实时动态网络 Dijkstra 算法，更好地模拟了实际情况下的突水水流蔓延模型。
- (2) 模型考虑了矿井水灾发生后的各个方面，包括突水的时间、巷道的水流传播、矿工逃生路径的设计等，确保了矿井突水灾害后各个环节的应对策略。
- (3) 通过动态调整矿工逃生路径和考虑不同突水点的情况，模型能灵活应对多种复杂情况，具备较强的适应性。
- (4) 在涉及到多种复杂情况（如多个突水点、矿工逃生路径设计等）时，模型的计算复杂度较高。尤其是在三维模型中，随着巷道网络规模的增大，计算量将大幅增加，可能导致处理时间的延长。
- (5) 由于时间有限，我们建立了四个模型，解决问题一到问题四，我们可以进一步以突水点数量和巷道高度为变量，从而建立一个更一般化的模型。

七、模型的推广

除了突水，矿井还可能面临其他类型的灾害（如瓦斯爆炸、火灾等），通过调整水流模拟参数，模型可被改造成适用于其他矿井灾害情境的应急响应工具。可跨行业应用，该模型的优化不仅能提升矿井安全管理的水平，还可以应用于城市地下管网、隧道施工、水下工程与救援、军事与国防等领域，进行水灾的模拟与逃生路径设计。

八、参考文献

- [1] 高风险岩溶隧道突水灾变演化机理及其应用研究. 李利平.山东大学,2009
- [2] 水压作用下防砂安全煤（岩）柱失稳突水溃砂机理研究. 李江华.中国矿业大学(北京),2016
- [3] 宁东某矿煤层顶板突水危险性研究. 湛会芹.中国地质大学(北京),2014
- [4] 煤层顶底板突水地质力学条件及其危险性研究. 王睿.中国矿业大学（北京）,2011
- [5] 基于多源数据的贫困度与自然灾害相关性评估. 赵习枝.华东师范大学,2019
- [6] 岩体水压致裂机理研究及在矿山突水中的运用. 刘洪磊.东北大学,2011
- [7] 煤矿区突（涌）水系统分析模拟及应用. 尹尚先.中国矿业大学（北京）,2002
- [8] 冲蚀诱发断层突水的流固耦合机理及应用研究. 王戈.河南理工大学,2020
- [9] 煤层顶板松散承压含水层渗流突涌特性及致灾机理与防治研究. 施小平.合肥工业大学,2015
- [10] 金属矿深竖井工程围岩渗透特征分析与突水风险预测研究. 张同钊.北京科技大学,2022
- [11] 北京月之暗面科技有限公司 kimi,k2 模型，深度搜索，2025-09-07

九、附录

附录一：

端点编号	水流到达时刻（分钟）	
P0000	264.57	
P0001	273.69	
P0002	290.08	
P0003	316.79	
P0004	169.96	
P0005	57.19	
P0006	54.54	
P0007	72.75	
P0008	50.97	
P0009	120.24	
P0010	341.74	
P0011	496.38	
P0012	592.13	
P0013	622.35	
P0014	545.22	
P0015	489.45	
P0016	378.3	
P0017	291.3	
P0018	284.49	
P0019	282.34	
P0020	286.04	
P0021	284.86	
P0022	290.87	
P0023	284.99	
P0024	295.03	
P0025	298.49	
P0026	248.59	
P0027	1067.72	
P0028	1118.86	
P0029	45.21	
P0030	56.39	
P0031	1176.63	
P0032	1170.13	

< > >|
端点水流到达时刻
巷道水流充满时刻
+

附录二:

	A	B	C
1	巷道编号	巷道充满水时刻 (分钟)	
2	H0001	50161.80	
3	H0002	50161.80	
4	H0003	50161.80	
5	H0004	50161.80	
6	H0005	50161.80	
7	H0006	50161.80	
8	H0007	50161.80	
9	H0008	50161.80	
0	H0009	50161.80	
1	H0010	50161.80	
2	H0011	50161.80	
3	H0012	50161.80	
4	H0013	50161.80	
5	H0014	50161.80	
6	H0015	50161.80	
7	H0016	50161.80	
8	H0017	50161.80	
9	H0018	50161.80	
0	H0019	50161.80	
1	H0020	50161.80	
2	H0021	50161.80	
3	H0022	50161.80	
4	H0023	50161.80	
5	H0024	50161.80	
6	H0025	50161.80	
7	H0026	50161.80	
8	H0027	50161.80	
9	H0028	50161.80	
0	H0029	50161.80	
1	H0030	50191.80	
2	H0031	50161.80	
3	H0032	50161.80	
4	H0033	50161.80	

< < > > 端点水流到达时刻 巷道水流充满时刻 +

附录三:

#	A	B	C	D
633	P0633	2459.45		
634	P0634	2285.47		
635	P0635	2482.36		
636	P0636	2525.70		
637	P0637	2533.58		
638	P0638	2766.41		
639	P0639	2097.98		
640	P0640	1937.50		
641	P0641	1551.48		
642	P0642	2671.99		
643	P0643	3789.72		
644	P0644	3188.72		
645	P0645	2727.24		
646	P0646	2643.70		
647	P0647	2505.93		
648	P0648	2216.62		
649	P0649	3853.66		
650	P0650	6925.95		
651	P0651	1015.20		
652	P0652	958.34		
653	P0653	923.22		
654	P0654	2148.89		
655	P0655	2295.41		
656	P0656	2000.53		
657	P0657	624.18		
658	P0658	836.68		
659	P0659	2129.82		
660	P0660	1995.48		
661	P0661	2408.26		
662	P0662	2354.75		
663	p3095	50227.15		
664	P389	3.05		
665				
666				

< > >>
端点水流到达时刻
巷道水流充满时刻
+

WP
□

附录四:

A	B	C
巷道编号	巷道充满水时刻 (分钟)	
H0644	50227.15	
H0969	24195.25	
H0968	23642.11	
H0967	23505.47	
H0966	23105.32	
H0965	23047.76	
H0964	22842.70	
H0963	22738.51	
H0962	22699.33	
H0975	22699.33	
H0961	22596.44	
H0974	22596.44	
H0041	11800.48	
H0051	11800.48	
H0040	11389.07	
H0050	11389.07	
H0039	11337.63	
H0049	11337.63	
H0054	11337.63	
H0038	11276.19	
H0048	11276.19	
H0053	11276.19	
H0037	11253.68	
H0047	11253.68	
H0052	11253.68	
H0036	11238.23	
H0046	11238.23	
H0035	11108.29	
H0045	11108.29	
H0034	11106.58	
H0044	11106.58	
H0033	11048.44	
H0043	11048.44	

端点水流到达时刻
巷道水流充满时刻

附录五：

附件一图形生成代码：

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import pandas as pd
import matplotlib.pyplot as plt

# 1. 读端点（跳过 3 行说明，手工给列名）
pts = pd.read_excel('附件 1.xlsx',
                    sheet_name='端点',
                    header=None,
                    skiprows=3,          # 跳过表头说明
                    usecols='A:D',
                    names=['端点编号', 'x', 'y', 'z'],
                    dtype={'端点编号': str})

# 去掉可能出现的空行
pts = pts[pts['端点编号'].notna()]
pts.set_index('端点编号', inplace=True)

# 2. 读巷道（同样跳过 3 行）
edges = pd.read_excel('附件 1.xlsx',
                      sheet_name='巷道',
                      header=None,
                      skiprows=3,
                      usecols='A:C',
                      names=['巷道编号', 'p1', 'p2'],
                      dtype={'巷道编号': str, 'p1': str, 'p2': str})
edges = edges[edges['p1'].notna() & edges['p2'].notna()]

# 3. 绘图
plt.figure(figsize=(12, 9))
for _, row in edges.iterrows():
    p1, p2 = row['p1'], row['p2']
    if p1 not in pts.index or p2 not in pts.index:    # 保险
        continue
    x1, y1 = pts.loc[p1, 'x'], pts.loc[p1, 'y']
    x2, y2 = pts.loc[p2, 'x'], pts.loc[p2, 'y']
    plt.plot([x1, x2], [y1, y2], color='steelblue', linewidth=0.8)

plt.scatter(pts['x'], pts['y'], s=2, color='black')
plt.gca().set_aspect('equal')
plt.title('巷道二维平面图')
plt.savefig('tunnel_2d.png', dpi=300)
plt.show()
```

附件二图像生成代码：

1. 读端点坐标 —— 跳过 3 行说明，自己起名

```

coordinates = pd.read_excel('文件路径.xlsx',          # <-- 换成你的实际文件名
                           sheet_name='端点',
                           header=None,
                           skiprows=3,
                           usecols='A:D',
                           names=['端点编号','x','y','z'])

# 2. 读巷道连接 —— 同样跳过 3 行说明
tunnels = pd.read_excel('文件路径.xlsx',
                        sheet_name='巷道',
                        header=None,
                        skiprows=3,
                        usecols='A:C',
                        names=['巷道编号','巷道端点 1','巷道端点 2'])

# 3. 建坐标字典（用我们新起的列名 'x','y','z'）
coordinates_dict = dict(zip(coordinates['端点编号'],
                             zip(coordinates['x'], coordinates['y'], coordinates['z'])))

# 4. 3D 画图
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# 画端点
for ep, (x, y, z) in coordinates_dict.items():
    ax.scatter(x, y, z, s=5)          # s=5 控制点的大小

# 画巷道线
for _, row in tunnels.iterrows():
    p1, p2 = row['巷道端点 1'], row['巷道端点 2']
    if p1 not in coordinates_dict or p2 not in coordinates_dict:
        continue                    # 防止缺号崩掉
    x1, y1, z1 = coordinates_dict[p1]
    x2, y2, z2 = coordinates_dict[p2]
    ax.plot([x1, x2], [y1, y2], [z1, z2], color='steelblue', linewidth=0.6)

ax.set_xlabel('X/m')
ax.set_ylabel('Y/m')
ax.set_zlabel('Z/m')
plt.title('3D Tunnel Layout')
plt.show()

附件一增加突水点+标记矿工+标记安全出入口代码：
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
绘制二维巷道图 + 自动判断 Source A2 所属巷道
"""

```



```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy.spatial.distance import cdist

# ----- 读表 -----
pts = pd.read_excel('附件 1.xlsx', sheet_name='端点', header=None, skiprows=3,
                    usecols='A:D', names=['id', 'x', 'y', 'z'], dtype={'id': str})
pts = pts[pts['id'].notna()].set_index('id')

edges = pd.read_excel('附件 1.xlsx', sheet_name='巷道', header=None, skiprows=3,
                      usecols='A:C', names=['eid', 'p1', 'p2'], dtype={'eid': str, 'p1': str,
                                'p2': str})
edges = edges[edges['p1'].notna() & edges['p2'].notna()]

# ----- 判断 Source A2 所属巷道 -----
source_A2 = np.array([[3760.40, 3808.33]])
best_eid, min_d = None, np.inf
for _, r in edges.iterrows():
    p1, p2 = r['p1'], r['p2']
    if p1 not in pts.index or p2 not in pts.index:
        continue
    seg = pts.loc[[p1, p2], ['x', 'y']].values
    d = np.min(cdist(source_A2, seg))
    if d < min_d:
        min_d, best_eid = d, r['eid']

print(f'Source A2 最接近巷道: {best_eid} (距离≈{min_d:.2f} m) ')

# ----- 绘图 -----
plt.figure(figsize=(12, 9))
for _, r in edges.iterrows():
    p1, p2 = r['p1'], r['p2']
    if p1 not in pts.index or p2 not in pts.index:
        continue
    x1, y1, x2, y2 = *pts.loc[p1, ['x', 'y']], *pts.loc[p2, ['x', 'y']]
    plt.plot([x1, x2], [y1, y2], color='steelblue', linewidth=0.8, zorder=1)

plt.scatter(pts['x'], pts['y'], s=2, color='black', zorder=2)
plt.scatter(*source_A2.T, marker='*', s=400, c='red', edgecolors='k',
            zorder=5, label=f'Source A2 ({best_eid})')
plt.scatter(5349.03, 4931.90, marker='*', s=400, c='red', edgecolors='orange',
            zorder=5, label='Source A1')

# 矿工 & 出口
miners = np.array([[5808.18, 5367.75], [5194.00, 4785.31], [6190.81, 3434.29]])
exits = np.array([[3252.16, 3326.63], [3173.10, 2819.97]])
plt.scatter(miners[:, 0], miners[:, 1], marker='o', s=80, c='yellow', edgecolors='k',
            zorder=4, label='Miners')

```

```
plt.scatter(exits[:, 0], exits[:, 1], marker='s', s=80, c='green', edgecolors='k',
            zorder=4, label='Safe exits')

plt.gca().set_aspect('equal', adjustable='box')
plt.title('Tunnel Layout')
plt.xlabel('x / m'); plt.ylabel('y / m')
plt.legend()
plt.savefig('tunnel_2d_layout.png', dpi=300)
plt.savefig('tunnel_2d_layout.pdf')
plt.show()
```

附件二生成图像+标注绿色出入口+标注黄色矿工+标注红色突水口代码:

```
# -*- coding: utf-8 -*-
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

# ----- 1. 读网 -----
coordinates = pd.read_excel(
    '附件 2.xlsx',
    sheet_name='端点',
    header=None, skiprows=3, usecols='A:D',
    names=['端点编号', 'x', 'y', 'z']
)
tunnels = pd.read_excel(
    '附件 2.xlsx',
    sheet_name='巷道',
    header=None, skiprows=3, usecols='A:C',
    names=['巷道编号', '巷道端点 1', '巷道端点 2']
)
coord = dict(zip(coordinates['端点编号'],
                  zip(coordinates['x'], coordinates['y'], coordinates['z'])))

# ----- 2. 定义特殊点 -----
# 绿色: 出入口
exits = np.array([
    [6336.99, 6073.22, 36.15],
    [6416.05, 6579.88, 8.69]
])

# 黄色: 矿工
miners = np.array([
    [4395.15, 4614.53, 6.59],
    [3398.34, 5965.56, 1.31],
    [3879.44, 4125.47, 6.22]
])
```

```

# 红色: 突水点 (A1 新增, A2 保留)
source_A1 = np.array([5883.14, 5643.35, 40.37]) # 新增 A1
source_A2 = np.array([4143.12, 4376.28, 6.33]) # 原 A2

# ----- 3. 空间点到线段最近距离 & 垂足判断 -----
def dist_point_to_segment(p, a, b):
    """返回点到线段 ab 的最短距离, 及垂足是否在线段内"""
    p, a, b = map(np.array, (p, a, b))
    ab = b - a
    ap = p - a
    t = np.dot(ap, ab) / np.dot(ab, ab) # 垂足参数
    if 0 <= t <= 1: # 垂足在线段内
        foot = a + t * ab
        return np.linalg.norm(p - foot), True
    else: # 取最近端点
        return min(np.linalg.norm(p - a), np.linalg.norm(p - b)), False

# 判断 A1 所属巷道
best_eid, best_p1, best_p2, min_d = None, None, None, np.inf
for _, row in tunnels.iterrows():
    p1, p2 = row['巷道端点 1'], row['巷道端点 2']
    if p1 not in coord or p2 not in coord:
        continue
    a, b = coord[p1], coord[p2]
    d, inside = dist_point_to_segment(source_A1, a, b)
    if d < min_d:
        min_d, best_eid, best_p1, best_p2 = d, row['巷道编号'], p1, p2

print(f'新增 Source A1 {source_A1} 所属巷道: {best_eid} ({best_p1}→{best_p2}, 距离≈{min_d:.2f} m) ')

# ----- 4. 画图 -----
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# 画巷道
for _, row in tunnels.iterrows():
    p1, p2 = row['巷道端点 1'], row['巷道端点 2']
    if p1 in coord and p2 in coord:
        (x1, y1, z1), (x2, y2, z2) = coord[p1], coord[p2]
        ax.plot([x1, x2], [y1, y2], [z1, z2], 'k-', linewidth=0.4, alpha=0.6)

# 特殊标记
ax.scatter(*exits.T, c='green', s=20, label='Exits')
ax.scatter(*miners.T, c='yellow', s=20, label='Miners')
ax.scatter(*source_A1, c='red', s=60, marker='*',
           label=f'Source A1 ({best_eid})') # ← 图例带巷道号
ax.scatter(*source_A2, c='red', s=60, marker='^', label='Source A2')

```



```

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.legend()
plt.title('Mine network with exits/miners/sources A1&A2')
plt.show()

```

经过删掉路径 MATLAB 代码:

% 定义图的边和权重

```

s = [000 002 003 005 005 005 005 010 011 012 014 015 016 018 019 019 019 022 022 022
025 027 006 006 031 033 033 033 037 038 038 038 041 041 041 041 046 046 046 047 047
050 050 050 054 054 054 028 028 059 059 059 063 063 063 067 067 067 071 071 071 072
072 075 075 075 078 078 078 082 084 086 088 088 088 091 092 092 093 093 094 094 096
096 097 097 100 100 100 101 101 104 104 102 108 039 039 110 110 110 114 116 118 119
119 119 023 020 122 123 123 123 121 127 127 130 130 130 133 133 133 137 137 137 140
140 141 141 141 134 134 146 146 146 147 147 148 148 150 154 154 149 149 157 157 161
161 155 163 163 162 165 165 164 158 160 151 166 166 169 169 167 171 172 172 175 175
175 179 176 181 181 181 182 185 186 186 186 190 191 188 193 193 196 198 143 143 199
199 144 145 145 136 202 202 202 203 203 204 204 206 208 208 211 211 211 210 212 212
213 214 214 215 216 216 217 219 219 218 221 221 220 224 224 224 225 225 226 226 228
034 227 229 229 230 232 232 233 236 236 236 237 237 238 238 240 242 242 243 244 244
245 246 246 247 250 250 250 251 251 252 252 254 256 256 257 258 258 259 260 260 261
262 262 263 264 264 265 266 266 267 268 268 269 270 270 271 272 272 273 274 274 275
276 276 277 278 278 058 279 279 279 074 280 280 281 282 282 283 285 285 284 287 287
286 286 290 290 288 291 291 292 293 293 294 297 297 297 295 296 298 299 299 300 302
302 301 304 304 303 306 306 305 309 309 309 308 307 310 234 311 312 314 316 318 319
319 319 322 322 322 320 320 320 325 326 326 326 330 330 330 331 331 007 336 336 336
336 340 340 340 343 343 344 344 348 348 348 349 349 350 350 350 346 353 353 354 352
352 351 347 337 337 341 359 360 362 362 362 363 363 138 207 200 368 369 115 371 373
373 373 085 374 374 375 168 168 168 378 378 379 379 381 076 076 040 040 043 387 387
387 390 390 390 109 383 384 380 380 382 391 391 392 392 392 112 396 396 131 132 132
397 399 399 398 079 400 401 401 401 080 081 081 083 376 376 377 408 410 410 411 412
412 413 416 321 197 173 173 418 418 419 170 421 422 422 422 423 423 424 424 426 430
430 428 428 429 431 431 432 433 433 434 435 435 436 437 437 438 440 440 443 444 439
445 445 445 441 446 446 447 448 448 449 450 450 451 452 452 453 454 454 455 456 456
457 458 458 057 459 459 459 460 460 464 464 464 466 465 249 462 462 463 467 467 468
469 469 470 253 222 222 223 471 471 472 473 473 474 414 475 475 475 476 476 477 477
479 481 481 482 483 483 484 485 485 486 487 489 491 491 492 492 496 493 493 494 497
497 498 406 499 499 495 500 500 501 502 502 503 504 504 505 506 506 507 508 508 403
509 509 511 511 511 404 510 510 512 513 513 514 515 515 394 516 516 395 517 517 518
519 519 520 152 478 478 480 521 521 522 524 524 156 523 523 525 527 527 528 364 103
103 529 529 529 060 060 061 061 069 531 531 070 107 239 239 241 532 532 533 534 534
535 536 536 537 538 538 539 540 540 541 542 542 543 062 544 544 544 545 547 547 548
546 550 550 550 550 552 552 552 549 549 554 554 554 553 035 385 098 099 099 231 024
024 556 556 556 560 561 561 561 562 562 558 008 564 564 564 565 565 565 566 566 526
568 569 570 571 551 551 572 572 572 575 575 575 576 576 577 577 578 579 579 580 581
581 582 583 583 584 585 585 586 587 587 588 559 559 563 589 589 590 591 591 592 593
593 594 594 594 596 596 596 574 597 599 600 372 601 602 603 604 604 117 605 606 606
606 124 608 608 608 611 611 611 613 613 613 616 617 618 619 620 621 622 622 622 614

```


614 623 623 624 624 624 051 626 626 627 627 627 630 630 630 184 631 631 631 631 632
632 632 628 628 629 633 633 633 634 634 635 635 635 567 567 638 317 317 640 640 640
641 641 642 643 644 645 194 646 647 636 637 315 648 648 648 648 323 323 649 649 649
598 126 126 120 370 370 370 612 612 610 001 142 142 142 129 052 052 652 652 652 651
651 053 339 030 367 367 367 361 361 639 654 654 654 654 064 064 065 655 655 442 332
345 345 333 333 333 334 658 328 195 205 026 048 004 049 049 049 386 388 656 357 357
659 659 660 660 661 661 662 425 555 017 095 090]; % 起始节点编号
t = [001 003 004 006 007 008 009 011 012 013 015 016 017 019 020 021 001 023 020 024
026 028 029 030 032 034 035 036 035 039 040 004 042 043 044 045 047 043 026 048 049
051 052 053 055 056 057 031 058 060 061 062 064 065 066 068 069 070 072 073 058 074
032 076 040 077 079 080 081 083 085 087 087 089 090 092 093 089 094 095 096 097 098
095 098 099 101 102 103 104 105 102 106 107 004 109 077 111 112 113 115 117 116 023
021 120 121 122 021 121 124 125 126 128 129 113 131 132 134 135 136 134 138 139 141
142 143 139 009 144 145 147 148 149 150 151 150 152 153 155 156 157 158 159 160 155
162 163 162 164 165 164 159 158 159 151 166 167 168 167 170 171 172 173 174 176 177
178 180 065 182 177 183 184 181 187 188 189 187 192 190 194 195 197 196 199 144 135
009 135 136 200 201 203 204 205 206 207 206 208 209 210 209 210 212 209 213 213 214
215 215 216 217 217 218 219 218 220 221 220 222 223 225 226 227 228 034 228 229 230
227 231 230 232 233 233 234 235 237 238 239 240 241 240 242 243 243 244 245 245 246
247 247 248 249 251 252 253 254 255 254 256 257 257 258 259 259 260 261 261 262 263
263 264 265 265 266 267 267 268 269 269 270 271 271 272 273 273 274 275 275 276 277
277 278 073 073 055 055 074 032 280 281 281 282 283 283 284 285 284 286 287 286 288
289 290 288 291 292 292 293 294 294 295 296 295 298 299 296 298 300 300 301 302 301
303 304 303 305 306 305 307 308 308 310 234 307 310 235 235 193 313 315 317 197 320
321 197 323 065 324 325 326 327 328 329 324 328 331 332 333 052 334 335 337 338 339
030 341 339 342 344 345 346 347 349 347 332 350 351 346 352 347 353 354 355 356 351
357 358 345 029 053 359 360 361 363 364 365 366 367 207 200 201 120 126 370 372 085
374 192 375 375 376 377 378 379 380 381 382 381 383 384 385 044 044 045 386 386 388
389 109 383 077 384 384 108 382 391 111 111 112 393 394 395 396 113 131 397 397 398
399 398 079 400 400 080 402 403 404 180 180 083 405 377 406 407 409 411 412 413 413
414 415 321 324 417 418 178 177 174 420 420 420 423 424 425 426 427 426 428 429 316
313 429 431 432 432 433 434 434 435 436 436 437 438 438 439 440 441 442 444 439 442
441 446 442 447 447 448 449 449 450 451 451 452 453 453 454 455 455 456 457 457 458
056 056 057 027 460 461 462 249 463 461 465 248 461 466 248 463 467 468 468 469 470
470 253 255 255 223 471 472 472 473 474 474 414 415 415 476 477 478 479 480 479 481
482 482 483 484 484 485 486 486 487 488 488 490 492 493 494 495 138 494 497 498 498
406 407 407 495 500 501 501 502 503 503 504 505 505 506 507 507 508 402 402 403 509
404 510 128 343 129 512 512 513 514 514 515 393 393 394 516 395 517 518 518 519 520
520 152 153 153 480 521 522 522 523 524 156 525 526 525 527 528 528 364 366 366 105
087 140 008 009 068 530 068 069 531 070 107 106 106 241 532 533 533 534 535 535 536
537 537 538 539 539 540 541 541 542 543 543 062 530 530 545 546 066 547 548 549 178
548 409 411 551 490 553 064 066 554 077 553 385 066 389 388 389 231 036 555 036 010
017 557 558 559 557 201 562 205 007 365 563 029 359 338 339 161 566 567 154 342 568
569 570 192 192 408 410 489 573 574 408 576 577 409 578 578 579 580 580 581 582 582
583 584 584 585 586 586 587 588 560 588 557 563 589 590 590 591 592 592 593 573 573
490 491 499 595 487 489 574 488 598 597 371 601 602 603 604 118 115 371 125 369 605
607 024 609 368 610 609 612 607 614 610 615 617 618 619 620 621 607 609 368 369 127
623 000 128 625 051 626 627 628 629 319 629 327 187 318 183 189 171 182 170 174 325
327 332 196 318 198 634 189 195 311 318 193 636 637 341 361 639 625 639 360 641 639
625 338 643 420 419 644 646 647 312 647 314 313 185 416 324 183 176 177 160 166 169

114 120 125 615 612 650 139 616 610 650 615 650 129 139 651 652 327 651 053 334 029
 653 338 335 335 335 342 365 638 642 195 329 655 087 656 329 655 329 089 555 443 657
 657 658 334 657 658 653 653 656 645 365 048 002 045 037 386 388 044 555 086 659 358
 660 661 662 358 662 425 427 427 389 018 091 105]; % 终止节点编号
 w = [51.491 150.905 817.473 15.205 87.949 49.367 356.194 873.604 540.9 170.728 315.083
 627.902 491.477 12.162 20.922 14.212 48.871 27.78 27.26 23.522 281.863 336.053 52.731
 10.531 42.745 117.606 13.959 34.809 32.133 26.548 19.465 24.8 814.98 49.746 26.671
 31.628 49.551 234.299 300.514 300.427 259.009 63.647 49.806 19.931 304.134 32.233
 30.794 326.371 36.338 29.136 70.482 70.195 49.725 27.461 157.578 69.473 29.832 70.414
 265.638 17.863 30.012 52.84 30.309 22.929 51.403 27.772 499.919 24.884 295.121 193.504
 65.064 34.653 33.94 22.704 25.275 40.544 49.329 52.632 59.429 40.962 40.453 50.644
 48.834 60.19 39.298 53.074 30.321 70.127 70.874 71.357 69.883 30.363 68.852 69.374
 99.873 18.814 25.546 881.387 25.967 1165.135 148.837 8.224 10.208 48.14 14.729 55.815
 21.375 14.179 20.319 31.063 57.458 36.869 35.803 51.025 272.514 258.293 26.774 190.546
 29.682 37.821 12.12 37.313 144.933 54.918 50.239 33.192 24.102 32.805 26.021 23.928
 26.611 25.894 299.624 203.296 300.344 302.307 25.796 301.021 299.529 54.952 955.372
 36.773 43.454 43.126 62.527 37.54 70.06 69.617 37.145 59.682 61.374 37.632 58.987 59.144
 36.912 24.576 49.062 50.248 343.058 24.71 27.457 103.845 49.794 28.12 24.81 17.055
 14.345 11.041 363.356 46.609 99.455 26.63 23.003 35.267 25.102 38.458 11.464 14.886
 11.485 18.29 37.474 19.92 26.552 17.18 11.255 25.821 26.216 27.025 23.185 25.861 26.147
 49.449 48.973 25.944 288.02 91.285 287.784 234.539 25.538 299.917 300.285 299.627
 25.519 25.138 300.199 299.431 300.168 24.474 294.608 328.663 42.479 225.076 191.387
 25.099 376.469 376.352 25.531 313.794 303.787 27.668 299.533 288.702 30.222 70.433
 69.449 69.873 69.895 29.673 269.492 270.072 30.846 81.669 29.441 365.547 365.272 29.397
 152.808 148.478 30.335 208.444 230.287 208.83 229.567 29.658 209.018 209.087 30.183
 213.108 214.063 30.509 208.84 208.498 30.304 200.357 199.311 29.564 203.738 215.904
 204.791 214.515 29.577 206.219 206.574 29.316 216.81 215.891 29.519 217.864 217.367
 31.466 211.145 211.813 31.057 212.895 212.895 31.333 207.611 207.785 30.934 143.052
 143.073 30.666 207.074 207.098 30.271 239.438 239.258 30.11 88.055 69.672 35.267 43.331
 61.701 30.053 78.171 94.78 34.299 27.82 7.85 30.496 52.371 49.807 50.039 30.277 140.709
 140.232 30.37 174.682 174.706 29.493 205.866 205.237 28.361 214.7 33.868 214.108 29.084
 418.612 418.725 28.501 208.031 207.791 29.169 843.64 843.583 209.902 28.254 209.309
 28.877 210.192 209.257 29.495 416.867 415.613 29.681 214.229 212.924 28.664 272.838
 272.53 29.061 639.786 640.347 197.006 28.645 502.541 28.992 198.425 505.746 30.064
 18.346 9.922 9.158 28.481 25.199 50.153 38.047 24.558 22.05 54.326 49.482 49.619 60.188
 38.163 60.55 103.812 50.081 50.313 18.211 14.491 19.531 17.335 19.09 29.622 50.336
 13.166 78.9 20.708 11.299 54.2 175.149 52.344 29.784 38.763 28.798 39.421 30.104 52.165
 30.632 30.598 29.537 29.524 39.639 49.852 50.046 25.456 25.458 30.671 396.751 396.096
 53.259 12.973 13.454 39 20.309 116.026 26.106 303.547 81.713 303.537 81.787 48.021
 16.086 25.472 26.363 26.377 26.878 39.494 30.202 76.923 35.534 63.133 26.425 64.478
 48.623 25.636 102.897 285.59 102.893 285.601 25.805 50.051 49.923 18.442 51.274 30.675
 25.866 25.399 28.949 49.678 41.924 25.316 80.53 18.983 80.758 26.278 49.486 25.265
 307.558 307.889 25.448 881.508 101.66 26.762 200.689 1164.819 25.854 258.861 190.118
 26.366 50.027 50.398 26.263 717.981 718.842 25.958 498.851 194.509 26.418 307.924
 295.486 25.685 1217.399 26.436 29.686 294.604 310.429 24.879 25.338 299.54 296.792
 26.432 299.184 302.759 26.854 23.366 23.452 25.914 22.412 22.17 28.86 52.998 27.156
 25.045 40.181 460.32 813.475 431.995 840.325 28.76 203.161 231.134 11.367 14.45 40.06
 218.106 191.704 28.997 30.168 29.08 28.946 159.538 160.122 30.298 290.565 290.362
 30.996 347.848 373.128 427.068 30.386 25.456 19.435 25.802 28.829 199.732 426.625
 200.023 29.627 300.267 299.56 29.601 300.1 299.916 29.597 400.129 400.178 29.068


```

718.729 701.698 34.649 44.559 62.247 30.446 62.43 78.924 34.287 47.927 43.249 28.874
63.339 278.698 207.59 278.774 25.642 18.626 117.619 19.551 24.623 29.981 28.65 27.413
27.325 28.513 187.932 187.634 29.799 208.124 208.84 30.284 25.928 28.823 29.347 25.936
266.731 267.126 25.448 304.483 303.552 25.869 25.846 357.917 595.792 358.265 595.808
25.34 239.646 240.323 25.49 296.391 296.082 25.624 298.138 297.63 25.608 347.709
347.643 24.804 18.016 27.59 200.041 200.034 717.66 19.412 27.364 299.788 299.947 26.81
299.972 300.184 25.823 27.379 275.727 275.908 27.253 308.59 308.232 27.147 272.177
297.325 36.293 95.669 70.533 26.334 30.246 30.59 26.397 195.592 307.171 26.016 302.733
272.226 436.81 52.131 303.435 24.979 299.365 300.699 27.38 51.325 51.863 26.35 101.167
200.984 24.844 251.186 250.51 25.789 303.899 304.066 25.955 99.495 100.118 26.848
26.118 303.804 304.162 25.845 296.202 216.836 27.14 79.154 1204.323 25.686 301.717
301.795 26.603 300.965 300.623 26.012 30.269 116.469 26.222 356.171 50.809 69.854
70.215 30.29 69.495 70.194 29.857 69.766 70.21 30.027 30.648 217.855 216.967 30.201
76.563 76.569 29.847 81.351 53.936 40.92 44.732 72.485 29.484 73.702 74.098 28.271
70.876 70.58 28.101 69.348 69.67 29.434 42.978 43.172 23.006 25.668 85.98 41.516 171.009
25.448 290.09 227.75 28.025 412.235 84.555 156.509 51.026 50.16 44.037 49.848 25.303
84.337 24.799 81.602 49.836 53.64 22.454 82.247 18.407 263.878 57.74 355.095 26.174
131.908 25.106 122.719 356.422 36.654 184.403 37.28 131.766 32.561 15.363 78.643 12.429
39.438 13.976 154.677 37.718 56.295 1125.771 1474.049 1116.547 286.491 34.89 303.395
214.918 27.307 411.451 416.165 296.829 26.117 299.664 297.821 298.749 25.352 366.047
368.537 24.388 89.305 86.356 25.522 196.427 182.569 28.711 45.804 59.783 24.562 65.228
79.643 46.692 28.39 32.442 24.905 77.872 77.43 24.798 299.201 300.965 34.748 580.691
578.666 25.456 402.274 399.977 318.236 24.648 264.155 407.113 25.35 264.113 141.274
137.164 58.503 114.794 30.44 8.936 61.948 30.418 45.088 114.298 32.172 7.647 29.662
20.984 34.037 28.495 39.888 26.589 20.699 26.49 10.367 124.302 50.249 92.69 11.901
11.247 13.132 10.357 12.102 23.49 39.275 28.38 20.514 300.836 49.928 304.149 300.626
55.346 50.062 41.601 42.07 20.555 36.232 37.607 13.794 49.318 23.456 48.458 12.73 22.916
24.69 26.664 76.037 49.614 37.693 50.57 105.605 25.033 17.329 19.918 34.797 48.984
28.786 13.669 42.084 7.364 8.666 10.243 76.8 21.749 133.686 38.048 48.71 16.745 77.204
32.095 9.077 18.555 223.546 49.723 19.933 92.295 19.712 7.603 7.836 10.511 7.885 9.294
22.128 13.169 24.497 25.409 29.314 27.889 48.748 25.192 50.566 56.707 50.149 30.153
48.625 32.858 49.684 665.377 23.171 49.965 32.779 85.832 665.02 50.391 50.074 466.307
19.447 105.65 8.909 49.588 18.061 52.761 18.396 50.485 22.674 72.832 236.615 20.834
25.227 29.146 14.128 81.161 50.297 29.305 88.246 29.672 29.035 49.834 49.932 82.054
265.966 18.851 20.15 9.435 14.241 18.469 14.184 9.734 10.398 17.308 74.074 44.691
356.717 50.207 283.621 19.747 65.652 50.258 30.071 49.715 40.956 70.343 398.636 27.327
29.207 55.689 71.855 399.095 33.232 70.931 55.549 28.383 50.734 38.424 51.813

```

75.019]; % 边的权重

s = s + 1;

t = t + 1;

% 创建图对象

G = graph(s, t, w);

% 为节点设置名称 (P0000, P0001, ..., P0662 等)

numNodes = numnodes(G);

nodeNames = cell(numNodes, 1);

for i = 1:numNodes

nodeNames{i} = sprintf('P%04d', i-1); % 节点编号从 0 开始

end

```

G.Nodes.Name = nodeNames;

% 找到节点索引
node660 = find(strcmp(G.Nodes.Name, 'P0660'));
node393 = find(strcmp(G.Nodes.Name, 'P0393'));
node659 = find(strcmp(G.Nodes.Name, 'P0659'));
node358 = find(strcmp(G.Nodes.Name, 'P0358'));
node662 = find(strcmp(G.Nodes.Name, 'P0662'));

% 删除从 P0660 到 P0659 和 P0358 的边
edges_to_remove = [];
for i = 1:numedges(G)
    [src, dst] = findedge(G, i);
    if (src == node660 && (dst == node659 || dst == node358)) || ...
        (dst == node660 && (src == node659 || src == node358))
        edges_to_remove = [edges_to_remove, i];
    end
end

% 创建修改后的图
G_modified = rmedge(G, edges_to_remove);

% 使用 Dijkstra 算法求最短路径
[path, dist] = shortestpath(G_modified, node660, node393);

% 显示结果
if isempty(path)
    fprintf('从 P0660 到 P0393 没有可达路径\n');
else
    fprintf('最短路径从 P0660 到 P0393:\n');
    path_names = G_modified.Nodes.Name(path);
    for i = 1:length(path_names)
        fprintf('%s ', path_names{i});
        if i < length(path_names)
            fprintf('-> ');
        end
    end
    fprintf('\n 总距离: %.2f 米\n', dist);
    % 可选: 可视化路径
    try
        figure;
        p = plot(G_modified, 'Layout', 'force');
        highlight(p, path, 'NodeColor', 'r', 'MarkerSize', 6);
        highlight(p, path(1), 'NodeColor', 'g', 'MarkerSize', 8);
        highlight(p, path(end), 'NodeColor', 'b', 'MarkerSize', 8);
        title(sprintf('从 P0660 到 P0393 的最短路径 (距离: %.2f 米)', dist));
    catch
        fprintf('可视化功能可能由于图太大而无法使用\n');
    end
end

```



```

end
end

% 显示删除的边信息
fprintf('\n 已删除的边:\n');
for i = 1:length(edges_to_remove)
[src_idx, dst_idx] = findedge(G, edges_to_remove(i));
src_name = G.Nodes.Name{src_idx};
dst_name = G.Nodes.Name{dst_idx};
fprintf('删除边: %s - %s\n', src_name, dst_name);
end

```

经典 Dijkstra MATLAB 代码实现:

```

% 定义图的边和权重
s = [000 002 003 005 005 005 005 010 011 012 014 015 016 018 019 019 019 022 022 022
025 027 006 006 031 033 033 033 037 038 038 038 041 041 041 041 046 046 046 047 047
050 050 050 054 054 054 028 028 059 059 059 063 063 063 067 067 067 071 071 071 072
072 075 075 075 078 078 078 082 084 086 088 088 088 091 092 092 093 093 094 094 096
096 097 097 100 100 100 101 101 104 104 102 108 039 039 110 110 110 114 116 118 119
119 119 023 020 122 123 123 123 121 127 127 130 130 130 133 133 133 137 137 137 140
140 141 141 141 134 134 146 146 146 147 147 148 148 150 154 154 149 149 157 157 161
161 155 163 163 162 165 165 164 158 160 151 166 166 169 169 167 171 172 172 175 175
175 179 176 181 181 181 182 185 186 186 186 190 191 188 193 193 196 198 143 143 199
199 144 145 145 136 202 202 202 203 203 204 204 206 208 208 211 211 211 210 212 212
213 214 214 215 216 216 217 219 219 218 221 221 220 224 224 224 225 225 226 226 228
034 227 229 229 230 232 232 233 236 236 236 237 237 238 238 240 242 242 243 244 244
245 246 246 247 250 250 250 251 251 252 252 254 256 256 257 258 258 259 260 260 261
262 262 263 264 264 265 266 266 267 268 268 269 270 270 271 272 272 273 274 274 275
276 276 277 278 278 058 279 279 279 074 280 280 281 282 282 283 285 285 284 287 287
286 286 290 290 288 291 291 292 293 293 294 297 297 297 295 296 298 299 299 300 302
302 301 304 304 303 306 306 305 309 309 309 308 307 310 234 311 312 314 316 318 319
319 319 322 322 322 320 320 320 325 326 326 326 330 330 330 331 331 007 336 336 336
336 340 340 340 343 343 344 344 348 348 348 349 349 350 350 350 346 353 353 354 352
352 351 347 337 337 341 359 360 362 362 362 363 363 138 207 200 368 369 115 371 373
373 373 085 374 374 375 168 168 168 378 378 379 379 381 076 076 040 040 043 387 387
387 390 390 390 109 383 384 380 380 382 391 391 392 392 392 112 396 396 131 132 132
397 399 399 398 079 400 401 401 401 080 081 081 083 376 376 377 408 410 410 411 412
412 413 416 321 197 173 173 418 418 419 170 421 422 422 422 423 423 424 424 426 430
430 428 428 429 431 431 432 433 433 434 435 435 436 437 437 438 440 440 443 444 439
445 445 445 441 446 446 447 448 448 449 450 450 451 452 452 453 454 454 455 456 456
457 458 458 057 459 459 459 460 460 464 464 464 466 465 249 462 462 463 467 467 468
469 469 470 253 222 222 223 471 471 472 473 473 474 414 475 475 475 476 476 477 477
479 481 481 482 483 483 484 485 485 486 487 489 491 491 492 492 496 493 493 494 497
497 498 406 499 499 495 500 500 501 502 502 503 504 504 505 506 506 507 508 508 403
509 509 511 511 511 404 510 510 512 513 513 514 515 515 394 516 516 395 517 517 518
519 519 520 152 478 478 480 521 521 522 524 524 156 523 523 525 527 527 528 364 103
103 529 529 529 060 060 061 061 069 531 531 070 107 239 239 241 532 532 533 534 534
535 536 536 537 538 538 539 540 540 541 542 542 543 062 544 544 544 545 547 547 548

```


546 550 550 550 550 552 552 552 549 549 554 554 554 553 035 385 098 099 099 231 024
 024 556 556 556 560 561 561 561 562 562 558 008 564 564 564 565 565 565 566 566 526
 568 569 570 571 551 551 572 572 572 575 575 575 576 576 577 577 578 579 579 580 581
 581 582 583 583 584 585 585 586 587 587 588 559 559 563 589 589 590 591 591 592 593
 593 594 594 594 596 596 596 574 597 599 600 372 601 602 603 604 604 117 605 606 606
 606 124 608 608 608 611 611 611 613 613 613 616 617 618 619 620 621 622 622 622 614
 614 623 623 624 624 624 051 626 626 627 627 627 630 630 630 184 631 631 631 631 632
 632 632 628 628 629 633 633 633 634 634 635 635 635 567 567 638 317 317 640 640 640
 641 641 642 643 644 645 194 646 647 636 637 315 648 648 648 648 323 323 649 649 649
 598 126 126 120 370 370 370 612 612 610 001 142 142 142 129 052 052 652 652 652 651
 651 053 339 030 367 367 367 361 361 639 654 654 654 654 064 064 065 655 655 442 332
 345 345 333 333 333 334 658 328 195 205 026 048 004 049 049 049 386 388 656 357 357
 659 659 660 660 661 661 662 425 555 017 095 090]; % 起始节点编号
 t = [001 003 004 006 007 008 009 011 012 013 015 016 017 019 020 021 001 023 020 024
 026 028 029 030 032 034 035 036 035 039 040 004 042 043 044 045 047 043 026 048 049
 051 052 053 055 056 057 031 058 060 061 062 064 065 066 068 069 070 072 073 058 074
 032 076 040 077 079 080 081 083 085 087 087 089 090 092 093 089 094 095 096 097 098
 095 098 099 101 102 103 104 105 102 106 107 004 109 077 111 112 113 115 117 116 023
 021 120 121 122 021 121 124 125 126 128 129 113 131 132 134 135 136 134 138 139 141
 142 143 139 009 144 145 147 148 149 150 151 150 152 153 155 156 157 158 159 160 155
 162 163 162 164 165 164 159 158 159 151 166 167 168 167 170 171 172 173 174 176 177
 178 180 065 182 177 183 184 181 187 188 189 187 192 190 194 195 197 196 199 144 135
 009 135 136 200 201 203 204 205 206 207 206 208 209 210 209 210 212 209 213 213 214
 215 215 216 217 217 218 219 218 220 221 220 222 223 225 226 227 228 034 228 229 230
 227 231 230 232 233 233 234 235 237 238 239 240 241 240 242 243 243 244 245 245 246
 247 247 248 249 251 252 253 254 255 254 256 257 257 258 259 259 260 261 261 262 263
 263 264 265 265 266 267 267 268 269 269 270 271 271 272 273 273 274 275 275 276 277
 277 278 073 073 055 055 074 032 280 281 281 282 283 283 284 285 284 286 287 286 288
 289 290 288 291 292 292 293 294 294 295 296 295 298 299 296 298 300 300 301 302 301
 303 304 303 305 306 305 307 308 308 310 234 307 310 235 235 193 313 315 317 197 320
 321 197 323 065 324 325 326 327 328 329 324 328 331 332 333 052 334 335 337 338 339
 030 341 339 342 344 345 346 347 349 347 332 350 351 346 352 347 353 354 355 356 351
 357 358 345 029 053 359 360 361 363 364 365 366 367 207 200 201 120 126 370 372 085
 374 192 375 375 376 377 378 379 380 381 382 381 383 384 385 044 044 045 386 386 388
 389 109 383 077 384 384 108 382 391 111 111 112 393 394 395 396 113 131 397 397 398
 399 398 079 400 400 080 402 403 404 180 180 083 405 377 406 407 409 411 412 413 413
 414 415 321 324 417 418 178 177 174 420 420 420 423 424 425 426 427 426 428 429 316
 313 429 431 432 432 433 434 434 435 436 436 437 438 438 439 440 441 442 444 439 442
 441 446 442 447 447 448 449 449 450 451 451 452 453 453 454 455 455 456 457 457 458
 056 056 057 027 460 461 462 249 463 461 465 248 461 466 248 463 467 468 468 469 470
 470 253 255 255 223 471 472 472 473 474 474 414 415 415 476 477 478 479 480 479 481
 482 482 483 484 484 485 486 486 487 488 488 490 492 493 494 495 138 494 497 498 498
 406 407 407 495 500 501 501 502 503 503 504 505 505 506 507 507 508 402 402 403 509
 404 510 128 343 129 512 512 513 514 514 515 393 393 394 516 395 517 518 518 519 520
 520 152 153 153 480 521 522 522 523 524 156 525 526 525 527 528 528 364 366 366 105
 087 140 008 009 068 530 068 069 531 070 107 106 106 241 532 533 533 534 535 535 536
 537 537 538 539 539 540 541 541 542 543 543 062 530 530 545 546 066 547 548 549 178
 548 409 411 551 490 553 064 066 554 077 553 385 066 389 388 389 231 036 555 036 010
 017 557 558 559 557 201 562 205 007 365 563 029 359 338 339 161 566 567 154 342 568
 569 570 192 192 408 410 489 573 574 408 576 577 409 578 578 579 580 580 581 582 582

583 584 584 585 586 586 587 588 560 588 557 563 589 590 590 591 592 592 593 573 573
490 491 499 595 487 489 574 488 598 597 371 601 602 603 604 118 115 371 125 369 605
607 024 609 368 610 609 612 607 614 610 615 617 618 619 620 621 607 609 368 369 127
623 000 128 625 051 626 627 628 629 319 629 327 187 318 183 189 171 182 170 174 325
327 332 196 318 198 634 189 195 311 318 193 636 637 341 361 639 625 639 360 641 639
625 338 643 420 419 644 646 647 312 647 314 313 185 416 324 183 176 177 160 166 169
114 120 125 615 612 650 139 616 610 650 615 650 129 139 651 652 327 651 053 334 029
653 338 335 335 335 342 365 638 642 195 329 655 087 656 329 655 329 089 555 443 657
657 658 334 657 658 653 653 656 645 365 048 002 045 037 386 388 044 555 086 659 358
660 661 662 358 662 425 427 427 389 018 091 105]; % 终止节点编号
w= [51.499 150.904 817.466 14.974 87.909 49.359 356.181 873.604 540.914 170.72 315.083
627.909 491.463 12.164 20.908 14.212 48.871 27.767 27.26 23.499 281.861 336.061 52.695
10.444 42.738 117.606 13.961 34.807 32.132 26.505 19.447 24.807 814.972 49.673 26.678
31.524 49.547 234.203 300.509 300.426 258.937 63.571 49.806 19.512 304.134 32.247
30.801 326.363 36.338 29.129 70.475 70.195 49.718 27.461 157.575 69.474 29.838 70.414
265.638 17.864 30.005 52.84 30.317 22.928 51.345 27.764 499.933 24.878 295.121 193.496
65.064 34.636 33.893 22.651 25.275 40.535 49.309 52.645 59.424 40.956 40.461 50.621
48.823 60.187 39.298 53.072 30.321 70.119 70.886 71.349 69.888 30.349 68.852 69.374
99.87 18.813 25.469 881.387 25.967 1165.142 148.83 8.224 10.208 48.133 14.7 55.809
21.375 14.179 20.305 31.067 57.451 36.883 35.802 51.024 272.521 258.279 26.767 190.546
29.665 37.806 12.12 37.313 144.915 54.915 50.214 33.204 24.101 32.724 25.953 23.932
26.572 25.456 299.617 203.289 300.326 302.321 25.796 301.028 299.528 54.941 955.356
36.773 43.446 43.124 62.392 37.541 70.062 69.612 37.139 59.681 61.358 37.627 58.994
59.157 36.915 24.576 49.055 50.255 343.045 24.71 27.444 103.842 49.793 28.103 24.817
17.056 14.352 11.034 363.356 46.567 99.459 26.63 23.002 35.259 25.02 38.451 11.470 14.9
11.492 18.283 37.481 19.926 26.552 17.178 11.166 25.768 26.04 27.004 23.17 25.847 26.065
49.439 48.965 25.932 288.019 91.289 287.783 234.539 25.524 299.91 300.278 299.641
25.519 25.145 300.192 299.438 300.154 24.474 294.618 328.666 42.478 224.926 191.262
25.075 376.454 376.311 25.53 313.788 303.761 27.675 299.533 288.695 30.227 70.432
69.455 69.878 69.885 29.668 269.491 270.071 30.853 81.655 29.429 365.554 365.279 29.381
152.791 148.471 30.342 208.444 230.28 208.83 229.56 29.651 209.03 209.08 30.168 213.108
214.075 30.509 208.833 208.484 30.311 200.364 199.318 29.557 203.731 215.904 204.798
214.515 29.584 206.219 206.567 29.302 216.81 215.891 29.519 217.864 217.36 31.459
211.145 211.821 31.057 212.902 212.902 31.333 207.611 207.785 30.934 143.052 143.067
30.659 207.059 207.098 30.264 239.445 239.258 30.11 88.055 69.672 35.267 43.324 61.694
30.053 78.171 94.787 34.309 27.835 7.85 30.51 52.379 49.807 50.046 30.271 140.709
140.218 30.363 174.682 174.706 29.5 205.866 205.236 28.353 214.701 33.86 214.108 29.091
418.619 418.718 28.509 208.031 207.79 29.172 843.633 843.59 209.897 28.247 209.307
28.869 210.186 209.25 29.486 416.874 415.612 29.674 214.236 212.924 28.664 272.845
272.531 29.068 639.786 640.347 196.992 28.645 502.555 28.999 198.418 505.753 30.07
18.331 9.915 9.155 28.48 25.188 50.153 38.012 24.509 22.058 54.306 49.468 49.607 60.096
38.149 60.533 103.79 50.073 50.221 18.204 14.498 19.537 17.333 18.746 29.562 50.329
13.14 78.893 20.676 11.247 54.213 175.099 52.35 29.791 38.763 28.798 39.428 30.104
52.158 30.632 30.591 29.529 29.531 39.632 49.851 50.043 25.456 25.458 30.671 396.751
396.11 53.259 12.972 13.377 38.989 20.242 116.008 25.942 303.535 81.708 303.443 81.705
47.997 16.086 25.402 26.37 26.37 26.878 39.493 30.202 76.915 35.54 63.133 26.419 64.492
48.615 25.627 102.89 285.596 102.9 285.594 25.811 50.043 49.907 18.391 51.28 30.571
25.859 25.399 28.945 49.677 41.91 25.248 80.543 18.983 80.758 26.245 49.472 25.266
307.551 307.896 25.44 881.515 100.625 26.648 199.609 1164.819 25.861 258.861 190.125
26.366 50.041 50.398 26.263 717.967 718.828 25.958 498.858 194.502 26.432 307.923


```

295.479 25.685 1217.392 26.426 29.693 294.516 310.358 24.892 25.339 299.554 296.792
26.421 299.177 302.759 26.849 23.359 23.427 25.91 22.412 22.182 28.852 52.992 26.856
24.744 40.181 460.314 813.483 431.988 840.332 28.76 203.161 231.127 11.375 14.45 40.05
218.106 191.704 29.003 30.168 29.08 28.953 159.531 160.129 30.291 290.565 290.354
30.996 347.845 373.139 427.068 30.386 25.456 19.435 25.816 28.808 199.717 426.625
200.005 29.627 300.27 299.563 29.601 300.1 299.916 29.611 400.122 400.178 29.061
718.743 701.705 34.649 44.545 62.233 30.446 62.444 78.924 34.28 47.92 43.249 28.881
63.339 278.691 207.597 278.774 25.642 18.619 117.612 19.544 24.623 29.988 28.651 27.421
27.31 28.519 187.932 187.634 29.806 208.131 208.847 30.276 25.924 28.83 29.361 25.939
266.731 267.119 25.444 304.482 303.552 25.861 25.838 357.917 595.779 358.261 595.809
25.267 239.622 240.317 25.371 296.388 296.018 25.579 298.13 297.607 25.606 347.652
347.574 24.802 18.016 27.589 200.012 200.006 717.585 19.419 27.363 299.755 299.917
26.824 299.874 300.087 25.823 27.379 275.673 275.861 27.246 308.573 308.222 27.147
272.174 297.316 36.293 95.676 70.533 26.334 30.232 30.583 26.404 195.592 307.177 26.023
297.23 272.24 436.765 52.11 297.262 24.737 298.506 298.66 25.931 51.306 51.346 26.35
100.441 199.631 24.851 250.416 249.753 25.788 300.371 300.522 25.962 99.487 100.111
26.837 26.058 303.797 304.152 25.729 296.207 216.782 27.138 79.14 1204.316 25.651
301.71 301.788 26.610 300.965 300.62 25.98 30.276 116.466 26.214 356.163 50.823 69.853
70.207 30.29 69.488 70.194 29.864 69.773 70.209 30.027 30.655 217.863 216.981 30.201
76.556 76.569 29.854 81.358 53.929 40.92 44.732 72.492 29.484 73.684 74.091 28.225
70.869 70.575 28.1 69.363 69.67 29.434 42.977 43.170 22.997 25.674 85.985 41.508 171
25.445 289.945 227.75 28.015 412.076 84.534 156.502 50.961 50.147 44 49.81 25.28 84.345
24.75 81.602 49.819 53.645 22.461 82.24 18.407 263.863 57.728 355.071 26.174 131.908
25.109 122.718 356.428 36.648 184.376 37.261 131.765 32.559 15.363 78.623 12.432 39.433
13.954 154.684 37.716 56.287 1125.778 1474.041 1116.554 285.413 24.419 303.261 214.918
27.297 411.458 415.875 296.7 26.114 299.614 297.669 298.693 25.343 366.057 368.541
24.388 89.287 86.353 25.517 196.420 182.548 28.703 45.803 59.766 24.558 65.209 79.63
46.697 28.369 32.441 24.897 77.813 77.43 24.714 298.632 300.937 25.814 580.376 578.665
25.427 402.274 399.963 318.155 24.648 264.115 406.828 25.35 264.052 141.281 137.171
58.511 114.794 30.447 8.942 61.955 30.418 45.078 114.292 32.172 7.647 29.663 20.987
34.046 28.461 39.868 26.588 20.703 26.483 10.353 124.302 50.241 92.69 11.908 11.248
13.139 10.35 12.095 23.477 39.248 28.338 20.521 300.829 49.928 304.156 300.619 55.345
50.062 41.601 41.733 20.562 35.982 37.581 13.753 49.301 23.456 48.441 12.737 22.909
24.683 26.674 76.037 49.612 37.421 50.34 105.561 24.924 17.261 19.914 34.754 48.937
28.755 13.67 42.09 7.363 8.657 10.254 76.753 21.763 133.673 38.046 48.722 16.74 77.203
32.102 8.995 18.562 223.558 49.723 19.761 92.279 19.725 7.595 7.832 10.514 7.876 9.288
22.128 13.121 24.472 25.341 29.3 27.879 48.734 25.199 50.58 56.707 50.13 30.16 48.631
32.858 49.672 665.383 23.178 49.953 32.779 85.833 665.011 50.373 50.066 466.3 19.05
105.646 8.902 49.595 18.061 52.737 18.013 50.484 22.674 72.814 236.613 20.639 25.227
29.119 14.127 81.151 50.286 29.254 88.158 29.686 29.035 49.834 49.924 82.026 265.957
18.851 19.821 8.679 14.249 18.095 13.7 9.734 9.718 17.308 74.071 44.684 356.716 50.144
283.635 19.717 65.65 50.257 30.071 49.635 40.956 70.276 398.636 27.327 29.207 55.689
71.841 399.095 33.225 70.924 55.556 28.383 50.742 38.431 51.806 75.013]

```

```
s = s + 1;
```

```
t = t + 1;
```

```
% 创建一个图形对象 G
```

```
G = graph(s,t,w);
```

```
% 绘制图形 G，并将边的权重添加到图形上
```



```

% G.Edges.Weight 表示图形对象 G 中所有边的权重值, 'EdgeLabel' 表示在图形上显示这些权重值
plot(G, 'EdgeLabel', G.Edges.Weight, 'linewidth', 2)
% 隐藏图形的坐标轴
set( gca, 'XTick', [], 'YTick', [] );

% shortestpath 函数计算从节点 9 到节点 8 的最短路径和路径长度, 并将路径和路径长度分别存储在 P 和 d 中
[P,d] = shortestpath(G, 18, 394)
% 在图形 G 中高亮显示最短路径
% highlight 函数高亮图形对象 myplot 中的路径 P, 'EdgeColor', 'r' 表示将路径颜色设置为红色。
myplot = plot(G, 'EdgeLabel', G.Edges.Weight, 'linewidth', 2);
highlight(myplot, P, 'EdgeColor', 'r')

% 计算图形 G 中任意两点之间的最短路径矩阵
D = distances(G)
% 输出 1 到 2 的最短路径
D(1,2)
% 输出 9 到 4 的最短路径
D(9,4)
% 找出图形 G 中距离节点 2 不超过 10 的所有节点
[nodeIDs,dist] = nearest(G, 2, 10)

```